May the Force *not* Be with you: Brute-Force Resistant Biometric Authentication and Key Reconstruction

Alexandra Boldyreva
Georgia Institute of Technology
Atlanta, Georgia, USA
sasha@gatech.edu

Deep Inder Mohan
Georgia Institute of Technology
Atlanta, Georgia, USA
dmohan@gatech.edu

Tianxin Tang
Eindhoven University of Technology
Eindhoven, Netherlands
ac.tianxin.tang@gmail.com

Abstract

The use of biometric-based security protocols is on the steep rise. As biometrics become more popular, we witness more attacks. For example, recent BrutePrint/InfinityGauntlet attacks showed how to brute-force fingerprints stored on an Android phone in about 40 minutes. The attacks are possible because biometrics, like passwords, do not have high entropy. But unlike passwords, brute-force attacks are much more damaging for biometrics, because one cannot easily change biometrics in case of compromise. In this work, we propose a novel provably secure Brute-Force Resistant Biometrics (BFRB) protocol for biometric-based authentication and key reconstruction that protects against brute-force attacks even when the server storing biometric-related data is compromised. Our protocol utilizes a verifiable partially oblivious pseudorandom function, an authenticated encryption scheme, a pseudorandom function, and a hash. We formally define security for a BFRB protocol and reduce the security of our protocol to the security of the building blocks. We implement the protocol and study its performance for the ND-0405 iris dataset.

1 Introduction

Motivation. Security of all practical cryptographic schemes is subject to brute-force (exhaustive-search) attacks. (The only brute-force-attack-resistant encryption scheme is the One Time Pad, but it comes with an impractical property of having the secret key as long as the messages.) Brute-force attacks are usually not a big concern because cryptographic secret keys of proper length have a lot of entropy and the exhaustive search is infeasible. However, this is only true if the keys are generated the intended way: picked uniformly randomly from large sets. However, in practice keys are often generated from users' passwords or biometrics for the convenience of key management. This way, users can re-compute and use the keys on any device, almost from scratch, without worrying about where and how to store the key. But in this scenario, brute-force attacks become a big concern because passwords and biometrics do not have high entropy.

Passwords are ubiquitous, but so are password-cracking tools. Passwords' vulnerabilities are well-known and there are various mechanisms to mitigate the risks. These include hashes, slow hashes, salts, rate-limiting, password managers, password-authenticated key exchange (PAKE) secure against offline brute-force attacks, asymmetric PAKE (aPAKE) that provides certain security even in case of server compromise and OPAQUE [33], currently considered for standardization, an aPAKE that provides even stronger security against pre-computation attacks. For even stronger protections, passwords can be stored in secure hardware, such as TPM, and some security can remain even if the TPM is compromised. This is an approach used by WhatsApp's end-to-end secure backup [16, 25]. We note, however, that despite *all* these techniques, in the strongest threat model, an adversary who compromises the server or the secure hardware can still perform brute-force attacks. This is usually considered unavoidable and acceptable. The idea is that after the compromise detection, the users change their passwords.

The use of biometric-based security protocols is on the steep rise. Fingerprints unlock phones and laptops, face scans permit passing airport security, palms are used for payments, etc. The biometric technology industry worldwide is projected to triple by 2030 [30] due to ease of use, (expected) stronger security, and increased reliability thanks to significant advancements in AI and computer vision technologies permitting automated recognition of human fingerprints, faces, irises, etc. It is symbolic that the Unicode Consortium just added the fingerprint emoji to its set.

As biometrics become more popular, we unfortunately witness more attacks. Recent BrutePrint/InfinityGauntlet attacks [13, 14] showed that after the attacker bypassed the rate-limiting for authentication attempts on an Android phone, it was not hard (40+ minutes) to brute-force fingerprints. After all, studies show that fingerprints do not have much more entropy than passwords [40].

Of course, there is a realization that biometric data has to be protected. Government regulations such as ISO/IEC 24745 international standard [31] and European Union's General Data Protection Regulation (GDPR) [27] regulate the protection of biometrics which they recognize as sensitive personal data.

Compared to passwords, the situation with attack protection for biometrics is worse. First, there are much fewer works on protecting biometrics than passwords[1]. Second, some techniques that work for passwords, such as OPRF (oblivious pseudorandom function) [12], strengthening employed by OPAQUE [33], WhatsApp's backup [16, 25], Hippo password manager [35], end-to-end password-based cloud storage [4], etc., are not easily applicable to biometrics.

And finally and most importantly, brute-force attacks are much more crucial for biometrics, because we can always replace compromised passwords but one cannot easily replace biometrics in case of compromise.

Our Focus. We aim to develop a provably secure Brute-Force Resistant Biometrics (BFRB) protocol for biometric-based authentication and key reconstruction that provides protection against brute force attacks even in case of server compromise.

Related Work. Fuzzy extractors [19] allow users to convert multiple noisy biometric readings w_i into the same cryptographic key K, with the help of error-correction and some "helper" data h. The

[1] Google Scholar gives almost 4 times more results for password security than for biometric security.

basic user authentication application mimicking password-based authentication would have the server storing $(h, f(K))$ where f is some one-way function, and h computed from the original biometric reading w. During authentication, likely over a secure channel, the server would get a scan w', would reconstruct K using h and will compare $f(K)$ with what it stored. Here, the server's storage is somewhat analogous (security-wise) to salted hashes of passwords. Simhadri et al. [36] build an authentication system based on fuzzy extractors dedicated to irises. Their system provides 32 bits of security, which is not very high. For fingerprints, it would be impractical as security would be even less. Very recently, Ahmad et al. [2] made improvements to the construction of [36] achieving an entropy of 42 bits for the True Accept Rate of 45%.

In these applications, if the user needs to reconstruct the key on various devices, then the helper data could be stored on the server and sent to the user when needed. The work [17] proposes a protocol to prevent servers from sending the helper data to attackers impersonating honest users since proper authentication relies on the helper data. Regardless, in case of the server compromise, the helper data is exposed to the attacker who will be able to brute force the biometrics, and compared to the passwords scenario, this is just too risky.

Password-based authenticated key exchange (PAKE) protocols allow parties knowing the password to agree on a secret key without the need for a secure channel. A number of works [10, 20, 22, 39] extend the notion of PAKE from passwords to noisy biometric strings. The protocols differ in security and efficiency but for all of them, similarly to the case with passwords, a malicious or compromised server yields brute-force attacks on users' biometrics. Again, this is too big of a risk.

A recent work proposed the BRAKE protocol [6] that claims to protect against brute-force attacks in case of a server compromise. However, [28] found that the claims are not valid and that BRAKE protocol does not provide the claimed protection.

Some protocols for biometric authentication consider 3-party settings [1, 8]. In [1], an additively-homomorphic encryption is used to register a biometric template to a service provider. Authentication happens at a terminal by homomorphic distance computation. The protocol requires the user to remember the secret key for the encryption scheme. In the applications we target, we assume that the user does not memorize any secrets, as we think that having to remember secret keys undermines the convenience of biometric protocols. Also, in their protocol, both the user and the terminal have direct access to biometric templates, while we consider settings where only the user knows the biometrics.

The protocol in [8] relies on two non-colluding servers and garbled-circuit-based secure multiparty computation. The user's biometric is secret-shared between the servers, and each server's compromise by itself does not leak information to the attacker. The protocol is implemented for DeepPrint [21] fingerprint templates and achieves state-of-the-art performance for the 2-server setting with online computation time of $\sim$ 50 milliseconds, reasonable communication costs of $\sim$ 300 KB, and a True Accept Rate (TAR) of 97.5%. At the same time, the protocol has several disadvantages. First, while they assume that the two servers are always non-colluding, the incentive for servers to collude may be too high since together they could immediately obtain all users' biometrics. This is especially concerning since both their servers have to be specialized to execute this 2PC protocol and are thus likely to be employed by the same company. Whereas our goal is to have one of the servers be a "strengthening" server, which may be used in tandem for other applications such as password strengthening. And even if both servers collude we still want them to perform the brute force attacks at the very least. Moreover, their protocol is secure in a weaker threat model and with stronger channel assumptions: they do not consider the malicious corruption of their authentication server, and they assume all communication channels to be secure. Next, the possibility of brute-forcing and the rate-limiting defense is not discussed in the paper. Finally, as the protocol only targets the goal of secure authentication, it is unclear if their techniques can be adapted to achieve our functionality goal of key reconstruction and secure storage. We provide more detailed performance and security comparisons after we discuss the details of our solution.

In the case of passwords, there are only a few works that offer protection against brute-force attacks by a malicious or compromised server. The HIPPO password manager [35] employs the help of an additional OPRF server that strengthens the passwords. As long as the OPRF server and the storage server do not collude, the adversary cannot brute-force passwords efficiently, even if it compromises either server (but not both). Here we believe that the non-colluding assumption is reasonable because the OPRF and storage servers are likely to be implemented by different companies. And even if both servers are compromised, the adversary still has to perform offline dictionary (brute force) attacks to recover the password. The protocol and its security are based on those for device-enhanced PAKE (DE-PAKE) [32]. The recent works [16, 25] analyze Whatsapp's backup storage protocol that utilizes secure hardware, TPM, playing the role of the helper server.

It is not immediately clear how these works could be extended to the case of biometrics. Moreover, even for passwords the aforementioned works do not fully answer the questions we target. The HIPPO password manager work [35] does not study the problem of authenticating to the storage server. DE-PAKE [32] is a PAKE in that the storage server knows the key (the strengthened password), which is not the case in our application. WhatsApp backup storage analyses [16, 25] do not consider a case when only the TPM is compromised. While it makes sense for their application, we consider a more general setting when only the helper server may be compromised.

Our contributions. We develop a novel provably secure Brute-Force Resistant Biometrics (BFRB) protocol for biometric-based authentication and key reconstruction that provides protection against brute force attacks even in case of server compromise. In order to provide the provable security analysis of our protocol, we first define the protocol's syntax and security.

Protocol's functionality. We start with specifying the functionality (syntax) of a BFRB protocol. A *Brute-Force Resistant Biometrics* (BFRB) protocol is run between three parties: a client C, the storage server $\mathcal{S}$, and the helper server $\mathcal{O}$. We assume clients have access to biometric readers. The storage server models some cloud storage services that clients can access via a secure channel such as TLS. The helper server models a separate service server, such as an OPRF server employed by numerous protocols such as Pythia or

OPAQUE [23, 33], but it can also model a secure hardware such as a TPM installed on the storage server. The helper server possesses a secret-public key pair, which we assume is honestly generated. We do not assume a secure channel between clients and the helper server. We will later consider the case of the authenticated channel from the helper server and discuss how this can help the efficiency of the protocol while achieving the same overall level of security.

Section 2 describes the input-output functionalities of the algorithms defining three phases, Setup, Register, and Retrieve, of a BFRB protocol. Briefly, in the Setup phase, the public system parameters and the keys for the servers are generated. In Register phase, a client with a biometric reading obliviously interacts with the helper server to obtain the biometric encoding. This biometric encoding is used to lock a client's secret message, such as a symmetric key, into a vault. The vault is stored on the storage server.

During the Retrieve phase, the client gets another biometric reading, which can be a noisy version of the original one, and repeats the same steps with the helper server to obtain a biometric encoding. This biometric encoding can be different from the one from the Register phase. The client then computes authentication data, with which it authenticates to the storage server, retrieves the vault, and unlocks it to retrieve the secret message.

The correctness requirement mandates that in honest executions of the protocol, a client could, with overwhelming probability, successfully retrieve the vault during Retrieve phase and unlock it to the same message that was locked during Register phase, as long as the biometric readings in both phases are close, according to some distance metric.

Security Definitions. Next, in Section 3, we formally define security of a BFRB protocol. We design a security model that captures very strong adversarial capabilities. We first consider scenarios when either server is corrupted.

An adversary corrupting the storage server gets to observe the communication between honest clients and the helper server. The attacker does not know the user biometric used in the Register phase, but it can know the distribution. The adversary can ask the same user to register multiple close biometrics, but this has to be done under different IDs. The adversary can observe the communication during the Retrieve phase as well, where it can specify how close the other biometric readings are to the challenge ones from the Register phase. In addition, the attacker can interact with the helper server as a client. Moreover, the adversary can mount MITM attacks impersonating the helper server to the honest clients. The adversary gets to see the resulting vaults and the authentication data. The adversary wins if it can violate message confidentiality or integrity. Namely, it wins if it gets any partial information about the messages locked in honest clients' vaults or if it can make an honest client successfully unlock a vault to a message different than what was locked during the Register phase.

An adversary corrupting the helper server learns the server's secret key. It gets to see honest clients' requests. Again, the adversary does not know the user biometric used in the Register phase, but it can know the distribution. The adversary can observe the requests during the Retrieve phase and specify how close the other biometric readings are to the challenge ones from the Register phase. The adversary can reply to all such requests, and it does not have to follow the protocol. The adversary does not get to observe the communication between honest clients and the storage server because we assume a secure channel there, but the vaults are created behind the scenes locking messages chosen by the attacker. The goals of the adversary are as follows. The attacker wins if it can come up with authentication data that would fetch a vault created by an honest client. (In this case, the adversary could unlock the vault by brute-forcing client's biometric.) The adversary also wins if it can make a client unlock a vault during the Retrieve phase to a message that is different from what the client chose during the Register phase.

We then consider a case where both servers are corrupted (colluding). While in this case, it is not possible to ensure security against brute-force attacks, we can aim to ensure that this is the best the attacker can do.

Finally, we discuss how the security definitions change if we add the assumption that the channel from the helper server to the client is authenticated.

Our Protocol and its Security. It is natural to try using an OPRF to strengthen the low-entropy biometrics, but the challenge is how to deal with its fuzziness. We do not see a way to build a BFRB protocol from a generic fuzzy extractor or fuzzy vault [34]. Instead, we rely on the specific sample-then-lock fuzzy extractor from [11].

The intuition behind the construction of this fuzzy extractor is as follows. Given a biometric feature vector $w \in \{0,1\}^n$, compute ℓ subsamples, which are restricted to k random indices from $[1, \ldots, n]$. Then the client's secret message can be encrypted under all subsamples, and the resulting ciphertexts will form a vault. For a later reading, the subsamples corresponding to the same projection indices are computed and all are used to attempt to decrypt the ciphertext. If the readings are close, then at least one subsample would be the same and decryption will succeed. This approach follows the construction of a locality-sensitive hash (LSH) for Hamming distance. For properly chosen parameters, false positives could be low. This means that for biometric scans (or feature vectors) that are not close, all subsamples are likely distinct, and hence the vault is protected.

This construction fits our protocol quite well because the users can apply the OPRF function to the subsamples described above. More precisely, during the Register phase, the user converts a biometric scan into a bitstring w and computes the subsamples $w_1, \ldots, w_n$, for a randomly chosen set of projection indices available as public parameters. (For simplicity, and because this is external to our protocol, we actually assume that the users' inputs are bitstrings, i.e., they are not the biometric readings, but the *feature vectors* extracted from the readings and converted to bitstrings. For the details of such extractions we refer the readers to [2].) Then the user interacts with the helper server by obtaining the keys $k_1 = OPRF_x(w_1), \ldots, k_\ell = OPRF_x(w_\ell)$, where x is the secret key of the helper server. Now the user can encrypt its secret message using some authenticated encryption SE: $c_1 = \mathsf{Enc}_{k_1[1]}(K), \ldots, c_\ell = \mathsf{Enc}_{k_\ell[1]}(K)$. Here $k_i[1] \leftarrow PRF(k_i, \langle 0 \rangle)$ is the PRF output of a bitstring of all 0s under the key k_i (using a keyed PRF like AES). All the ciphertexts are sent to the storage server as the vault V. The client

also sends $(k_1[2], \ldots, k_\ell[2]) \leftarrow PRF(k_1, \langle 1 \rangle), \ldots, PRF(k_\ell, \langle 1 \rangle)$ to the storage server (via a secure channel) as the authentication data.

During the Retrieve phase, the user obtains a new reading w' and follows the same steps with the helper server to obtain $k'_1, \ldots, k'_\ell$. Now the user will authenticate to the server using $k'_1[2]$, $\ldots$, $k'_\ell[2]$ as *ad*. If the authentication succeeds (at least one $k_i[2] = k'_i[2]$), the server will send the user the corresponding vault, which c_i. If the biometric scan (or feature vector) was close to the one used during registration, then at least one $k'[i]$ will match $k[i]$, and hence the user will be able to decrypt the corresponding vault to obtain K from $c[i]$.

While the protocol described above "works" for the sake of correctness, it is not secure. The malicious helper server can compute several keys $k_1, \ldots, k_q$ from its secret key and subsamples $w_1, \ldots, w_q$, and test them by trying to authenticate to the storage server. Even an outsider adversary can mount a brute-force attack by repeatedly sending blinded requests to the helper server and then trying to authenticate to the storage server.

A solution, known from solving a similar problem for password strengthening, is to use a *partially* oblivious PRF (*P*OPRF) and rate-limiting. With POPRF, there is a public part of the client's input which is signed, called a *tag*. In our protocol that would be the client's identity id. Each server would implement rate-limiting, aborting after a certain number of continuous requests per time period from the same id. We actually will require an additional property that POPRF is *verifiable* (VPOPRF). Section 5 describes the protocol in detail.

Next, we analyze the security of our protocol. We show that it provably provides message privacy and integrity against the malicious helper server and (separately) against the malicious storage server assuming the standard computational assumptions on the building blocks. The brute-force attacks (brute-forcing subsamples w_i) are not feasible in either case due to rate-limiting enforced by the honest server. If both servers collude though, brute-force attacks are possible, but we prove that they are necessary.

Recall that our setting assumes the secure channel between clients and the storage server, but does not assume a secure or authenticated channel between the clients and the helper server. Instead, we rely on the verifiability of the POPRF with honestly generated keys.

However, in our protocol, to achieve an acceptable accept rate and security level, we need to apply a verifiable POPRF to thousands of inputs for each Retrieve phase. As we show below, the use of the state-of-the-art verifiable POPRF, 3HashSDHI [37], will cost tens of seconds under single-threaded execution.

We observe that in the setting where there is an authenticated channel from the helper server to the client, we do not need to rely on the verifiability of the POPRF during the Retrieve phase. The authenticated channel will prevent a malicious storage server from impersonating the helper server. If the corrupted helper server provides incorrect values to the client, the client can detect this since the authentication to the storage server will fail. In this case, the client can take measures. Even with correctness errors, an honest client's probability of not being able to authenticate to an honest storage server beyond three attempts becomes very small, meaning corruptions will be detected. Note that we still need verifiability during the Register phase to ensure that vaults and the authentication data are created correctly.

Instantiating BFRB and the Implementation Results. We instantiate our BFRB protocol with 3HashSDHI VPOPRF [37], SHA-512 hash, AES-256 PRF and AES-256 with the GCM-SIV mode of operation as authenticated encryption. For extracting binary features from iris images, and generating subsamples from features, we use the algorithms from the `IrisLock` [2] fuzzy extractor. Their CNN-based heterogeneous feature extractor is trained on iris images from the ND-0405 dataset [9]. For subsample extraction, we use their ζ-sampling algorithm to fix projection indices. The vault generation procedure for both `IrisLock` and our BFRB protocol does not impact protocol correctness post subsample generation. Therefore, the correctness of our protocol follows directly from the correctness analysis of `IrisLock`. Since our protocol's security does not rely on brute-force complexity (except in the case of both servers' compromise), our optimal parameters for ζ- sampling differ from those chosen by `IrisLock` as we trade worse subsample min-entropy for a reduction in correctness error. The BFRB protocol has a best-case True Accept Rate of 87%, compared to just 45% for `IrisLock`. Instantiation details are discussed in Section 7.

Our protocol's security relies on both honest storage and helper servers deploying rate-limiting. Choosing projection indices corresponding to 33-bit subsample min-entropy and setting an hourly limit of 10, and a monthly limit of 300, attackers against BFRB would require roughly 6 years to achieve an advantage close to 1. We can further upgrade this (at the cost of correctness error) by choosing projection indices corresponding to 42-bit subsample min-entropy, which with these rate limits would take about 3000 years.

To compare with [8], while DeepPrint fingerprint templates allow for an almost perfect True Accept Rate of 97.5%, the tradeoff is a poor False Accept Rate at 0.01%. This makes the estimated template min-entropy just 13.3 bits[2], which is unsuitable for high-security applications. While they do not discuss brute-force security and rate-limiting in their work, we imagine that this extreme entropy constraint will make it impossible to define reasonable rate-limits to claim brute-force resistance.

For performance analysis, we implement the BFRB protocol in Rust, and perform benchmarks on both single-thread and multi-threaded performance, while simulating overheads assuming cellular networks (100 Mb/s) and gigabit Ethernet (1 Gb/s). The total execution times for authentication and key retrieval vary from $\sim$ 40 seconds in single-thread execution in a cellular network, to $\sim$ 5.5 seconds in parallel execution in a gigabit network. These times are further reduced to $\sim$ 25 seconds and $\sim$ 3.5 seconds respectively by skipping POPRF verification, i.e., assuming an authenticated channel to the helper server. Detailed performance analysis can be found in Section 8.

While the protocol in [8] provides much better performance (400 ms offline; 50 ms online vs our 3.5 seconds for best case performance), communication cost (35 MB offline; 300 KB online vs our 70 MB), and permits lightweight clients, the main advantage of our protocol is stronger security guarantees, and more formal

[2] $H_\infty \leq -\log_2(FAR) = -\log_2(0.0001) = 13.29$

treatment of brute-force attacks, as discussed in the related work section.

Concurrent work. After we completed our work, the IrisLock protocol has been updated[3]. Our experiments, however, were based on their previous code version and results[4]. IrisLock authors have introduced a new feature extractor and a new dataset, claiming to achieve 105 bits of entropy with a 0.91 TAR, as compared to 42 bits of entropy at 0.45 TAR in their previous version. We emphasize that our approach is for mitigating brute-force attacks on arbitrary low to medium min-entropy authentication data. This makes our protocol suitable for a wide range of biometric applications.

1.1 Notation

For every $x \in \mathbb{N}$, we use $\langle x \rangle$ to denote its string representation. For every bit string w in $\{0,1\}^*$, we may, for convenience, represent it either as a single element in $\{0,1\}^*$ or in its list form. In the list representation, to extract a single bit, we use $w[i]$, where i is the index. For every map $\mathsf{m} : \{0,1\}^* \to \{0,1\}^*$, we use $\mathsf{m}[\mathsf{id}]$, where id is the key field, to retrieve its associated value. Similarly, for two dimensional maps $\mathsf{m} : \{0,1\}^* \times \{0,1\}^* \to \{0,1\}^*$, we use $\mathsf{m}[\mathsf{id1}, \mathsf{id2}]$. We use the notation $\mathsf{m} \leftarrow \{\}$ to initialize m to a python-style dictionary. We use $\langle 0 \rangle$ and $\langle 1 \rangle$ to represent bitstrings of all 0s and all 1s respectively. Since many algorithms utilize states, for simplicity, we use the same notation st before and after assignment, though the state st is overridden. We assume the counters such as q_{id} for every id are all initialized to 0. For every $a, b \in \mathbb{N}$, we use $[a, b]$ to denote the discrete range $a, a+1, \ldots, b$.

2 BFRB Functionality (Syntax)

Here we provide the input-output functionality of our protocol (its syntax). But first, we define some terms for biometrics.

Biometric Readings Closeness. Biometric-based protocols need to distinguish biometric readings from different sources. In practice, this usually involves measuring the distance between two readings according to some metric and checking if it is within a certain threshold. If within the threshold, the readings are considered close and likely from the same source; otherwise, they are treated as from different sources. We formalize this process as follows.

We assume that the biometric space $\mathbb{W}$ is a metric space with distance metric δ and that the *triangle inequality* holds in $\mathbb{W}$ with respect to δ. We then define an algorithm to decide the closeness of biometric readings, denoted as $\mathsf{close}_\delta(w, w', \tau)$. The algorithm takes as input two biometric readings $w, w' \in \mathbb{W}$ and a threshold parameter τ. It outputs 1 iff $\delta(w, w') \le \tau$; otherwise, it outputs 0.

BFRB Syntax. A *Brute-Force Resistant Biometrics* (BFRB) protocol involves three parties: a client C, a storage server $\mathcal{S}$, and a helper server O. Let $\lambda \in \mathbb{N}$ be the security parameter. Let $\mathcal{W}$ be a biometric sampler sampling readings (or feature vectors) in $\mathbb{W}$. $\mathcal{W}$ takes an input, which is either $w \in \mathbb{W}$ or $\perp$. In the former case, it samples a close reading, to model a different reading of the same biometric: $w' \leftarrow\!\!\$\ \mathcal{W}(w)$, where $\delta(w, w') \le \tau$. In the latter case, it is a fresh reading in $\mathbb{W}$: $w \leftarrow\!\!\$\ \mathcal{W}(\perp)$.

[3] https://eprint.iacr.org/archive/2024/100/20250415:013018
[4] https://eprint.iacr.org/archive/2024/100/20241106:010157

A BFRB protocol consists of three phases: Setup, Register, and Retrieve.

Setup. This phase is executed once in the beginning.

- $pp \leftarrow \mathsf{ParamGen}(1^\lambda)$ generates the public parameters given the security parameter. We assume these become publicly available to all parties.
- $(pk_\mathrm{o}, sk_\mathrm{o}; st_\mathrm{o}) \leftarrow\!\!\$\ \mathsf{InitO}(pp)$ is run by the helper server O to generate its keys and initialize its persistent state.
- $st_\mathrm{s} \leftarrow \mathsf{InitS}(pp)$ is run by the storage server $\mathcal{S}$ to initialize its persistent state. This state st_s contains an empty database DB, denoted as $st_\mathrm{s}.\mathsf{DB}$.

Register. This phase is executed only once per client. In this phase, C derives an encoding of its biometric input with the helper server O, and locks a secret message of its choice into a vault using that encoding. The vault is then sent to the storage server $\mathcal{S}$ for storage. The Register phase consists of the following algorithms.

- $(\overline{w}; st_\mathrm{c}) \leftarrow\!\!\$\ \mathsf{BlindBio}(pp, pk_\mathrm{o}, \mathsf{id}, w)$ is run by the client C, it takes as input the public parameters pp, the helper server's pk_o, the client's $\mathsf{id} \in \{0,1\}^*$, and the biometric reading $w \in \mathbb{W}$. It returns a blinded version of the biometric reading $\overline{w}$, and a temporary state st_c. $\overline{w}$ and id are sent to O.
- $(\overline{w_\mathrm{enc}}; st_\mathrm{o}) \leftarrow \mathsf{EncBio}(pp, sk_\mathrm{o}, \mathsf{id}, \overline{w}; st_\mathrm{o})$ is run by the helper server O. It takes as input the public parameters pp, its secret key sk_o, its persistent state st_o, the client's id, and the blinded biometric $\overline{w}$. It updates its persistent state st_o and outputs the blinded biometric encoding $\overline{w_\mathrm{enc}}$, which is sent to C.
- $w_\mathrm{enc} \leftarrow \mathsf{UnblindBio}(pp, pk_\mathrm{o}, \overline{w_\mathrm{enc}}; st_\mathrm{c})$ is run by the client C to unblind the blinded biometric encoding $\overline{w_\mathrm{enc}}$ using its temporary state st_c. The temporary state st_c is then removed.
- $V \leftarrow \mathsf{GenVault}(pp, \mathsf{id}, w_\mathrm{enc}, m)$ is run by the client C. It takes as input the public parameters pp, the biometric encoding w_enc, and the client's chosen secret message $m \in \{0,1\}^*$ to generate the vault V. V is sent to the storage server $\mathcal{S}$.
- Finally, the storage server $\mathcal{S}$ adds the vault V under the client's id to the database DB in its persistent state st_s: $st_\mathrm{s}.\mathsf{DB}[\mathsf{id}] \leftarrow V$.

Retrieve. In this phase, the client C uses a new biometric reading to securely retrieve the vault from the storage server $\mathcal{S}$, unlock it, and obtain its locked message. This phase can be executed as many times as needed. The first three algorithms are identical to those from Register stage.

- $(\overline{w'}; st'_\mathrm{c}) \leftarrow\!\!\$\ \mathsf{BlindBio}(pp, pk_\mathrm{o}, \mathsf{id}, w')$.
- $(\overline{w'_\mathrm{enc}}; st_\mathrm{o}) \leftarrow \mathsf{EncBio}(pp, sk_\mathrm{o}, id, \overline{w}; st_\mathrm{o})$.
- $w'_\mathrm{enc} \leftarrow \mathsf{UnblindBio}(pp, pk_\mathrm{o}, \overline{w'_\mathrm{enc}}; st'_\mathrm{c})$.
- $ad \leftarrow \mathsf{GenAuthData}(pp, \mathsf{id}, w'_\mathrm{enc})$ is run by the client C to derive authentication data ad from the biometric encoding w'_enc and public parameters pp. The pair (id, ad) is sent to the storage server $\mathcal{S}$.
- $(V/\perp; st_\mathrm{s}) \leftarrow \mathsf{Authenticate}(pp, \mathsf{id}, ad; st_\mathrm{s})$ is run by the storage server $\mathcal{S}$ to fetch and authenticate the corresponding vault V from its database DB under the client id. This algorithm outputs a vault V or $\perp$ if the authentication fails. The storage server $\mathcal{S}$ updates its st_s and sends $V/\perp$ to the client C.
- $m \leftarrow \mathsf{UnlockVault}(w'_\mathrm{enc}, \mathsf{id}, V)$ is run by the client C to retrieve its locked message.

Correctness. Correctness for a brute-force resistant biometric protocol BFRB, with threshold τ specified on distance metric δ is defined by experiment $\mathbf{Exp}^{\text{correctness}}_{\text{BFRB},\delta,\tau}$, presented in Figure 1. Even though correctness is about honest protocol executions, the use of an adversary helps to quantify it over all possible inputs. The adversary can run the protocol (honestly) with any input biometrics, ids, and messages of its choice. The adversary wins if it runs Retrieve and Register phases with close biometrics but fails to recover the original message. Correctness means this cannot happen except with a small probability.

We say that the scheme BFRB is correct with error χ if for all adversaries $\mathcal{A}$

$$\Pr[\mathbf{Exp}^{\text{correctness}}_{\text{BFRB},\delta,\tau}(\mathcal{A}) = 1] \leq \chi \ .$$

$\mathbf{Exp}^{\text{correctness}}_{\text{BFRB},\delta,\tau}(\mathcal{A})$

1 $pp \leftarrow \text{ParamGen}(1^\lambda)$
2 $(pk_o, sk_o; st_o) \xleftarrow{\$} \text{InitO}(pp)$
3 $st_s \xleftarrow{\$} \text{InitS}(pp)$
4 $MT \leftarrow \{\}$; win $\leftarrow 0$
5 $\mathcal{A}^{\text{OracRegister,OracRetrieve}}(pp, pk_o)$
6 **return** win

Oracle OracRegister(id, w, m)

7 **if** st_s.DB[id] $\neq \perp$ **then return** $\perp$
8 $(\overline{w}; st_c) \xleftarrow{\$} \text{BlindBio}_C(pp, pk_o, \text{id}, w)$
9 $(\overline{w_{\text{enc}}}; st_o) \leftarrow \text{EncBio}_O(pp, sk_o, \text{id}, \overline{w}; st_o)$
10 $w_{\text{enc}} \leftarrow \text{UnblindBio}_C(pp, pk_o, \overline{w_{\text{enc}}}; st_c)$
11 $V \leftarrow \text{GenVault}_C(pp, \text{id}, w_{\text{enc}}, m)$
12 $st_s.DB[\text{id}] \leftarrow V; MT[\text{id}] \leftarrow m$

Oracle OracRetrieve(id, w')

13 **require** st_s.DB$[id] \neq \perp$
14 $m \leftarrow MT[\text{id}]$
15 $(\overline{w'}; st'_c) \xleftarrow{\$} \text{BlindBio}_C(pp, pk_o, \text{id}, w')$
16 $(\overline{w'_{\text{enc}}}; st_o) \leftarrow \text{EncBio}_O(pp, sk_o, \text{id}, \overline{w'}; st_o)$
17 $w'_{\text{enc}} \leftarrow \text{UnblindBio}_C(pp, pk_o; st'_c)$
18 $ad \leftarrow \text{GenAuthData}_C(pp, \text{id}, w'_{\text{enc}})$
19 $(V'; st_s) \leftarrow \text{Authenticate}_S(pp, \text{id}, ad; st_s)$
20 $m' \leftarrow \text{UnlockVault}_C(w'_{\text{enc}}, \text{id}, V')$
21 **if** $\text{close}_\delta(w, w', \tau) = 1$ **then**
22 **if** $(m \neq m')$ **then** win $\leftarrow 1$
23 **return** win

Figure 1: Experiment defining correctness of BFRB protocol.

3 BFRB Security Definitions

Let $\lambda \in \mathbb{N}$ be the security parameter. Let $\mathcal{W}$ be a biometric sampler. Let BFRB be a BFRB protocol.

Since we assume that the helper O and storage S servers do not collude, we first consider cases when only one of the two servers is malicious. Namely, we consider an adversary $\mathcal{A}$ who

- $\mathcal{A}$ corrupts the storage server S or
- $\mathcal{A}$ corrupts the helper server O.

For each case, we consider two adversarial goals: confidentiality and integrity. We will also consider the case of colluding malicious servers. While we cannot prevent brute-forcing in this case, we can still expect that this is all the attacker can do.

We assume that the communication channel between C and S is authenticated and secure. However, we do not make any such assumptions about the channel between C and O, meaning adversary $\mathcal{A}$ can snoop/control all communications between these two parties, and also mount man-in-the-middle attacks. Later, we will show that if the communication from O to C is authenticated, then we can rely on a weaker primitive and achieve better efficiency. We now study each case in detail.

3.1 Security in case of corrupt S

We present the game-based security definition for the case where the adversary controls the storage server S. The security experiment $\mathbf{Exp}^{\text{sec-cor-}S}_{\text{BFRB},\mathcal{W}}(\mathcal{A})$ is defined in Figure 2. We capture both goals of message confidentiality (indistinguishability) and integrity in the same experiment.

The experiment flips bit b, samples challenge biometric w^* (lines 1,2), and generates public parameters pp and keys for honest O server (lines 7,8). The adversary is given the public parameters and O's public key. It is given access to several oracles.

Oracle Blind lets $\mathcal{A}$ observe the honest clients' (blinded) requests to server O. $\mathcal{A}$ gets to choose the client ids. The oracle records the clients' states, which usually contain the blinding randomness (line 15).

Oracle OEncBio lets $\mathcal{A}$ observe server O's blinded biometric encoding response to either the honest clients' requests obtained from the Blind oracle or requests of $\mathcal{A}$'s choice (line 17).

Oracle LRVault lets the adversary to see clients' vaults created during Register stage. The adversary can select the id and the blinded biometric encoding. The latter could be obtained from OEncBio oracle to model the honest server O's response or be newly created by $\mathcal{A}$ to model $\mathcal{A}$ impersonating server O (playing the MITM between the client and the helper server). In either case, $\mathcal{A}$ must provide the query index so the experiment can obtain the client's state necessary to unblind the blinded biometric encoding (lines 21,22). We allow one vault per id. The adversary can have some partial information about the messages locked in the vaults; this is captured via the standard message indistinguishability approach (lines 24,25).

Oracle GetAuth lets the adversary to see the authentication data created during Retrieve stage. Similarly to LRVault oracle, the adversary provides the id and the blinded biometric encoding. The latter could be obtained from OEncBio oracle to model the honest server O's response or be newly created by $\mathcal{A}$ to model $\mathcal{A}$ impersonating server O. In either case, $\mathcal{A}$ must provide the query index so the experiment can obtain the client's state necessary to unblind

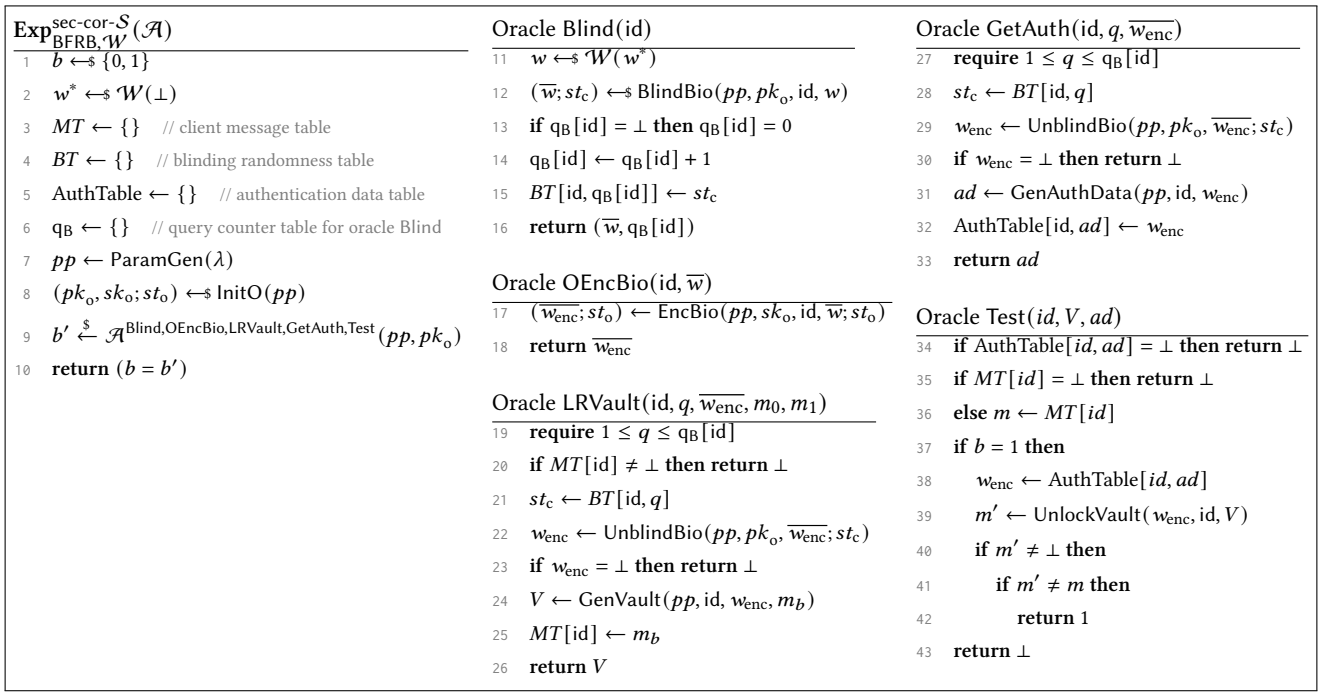

Figure 2: Experiment $\mathbf{Exp}^{\text{sec-cor-}\mathcal{S}}_{\text{BFRB},\mathcal{W}}(\mathcal{A})$ for client message indistinguishability and integrity for corrupted $\mathcal{S}$.

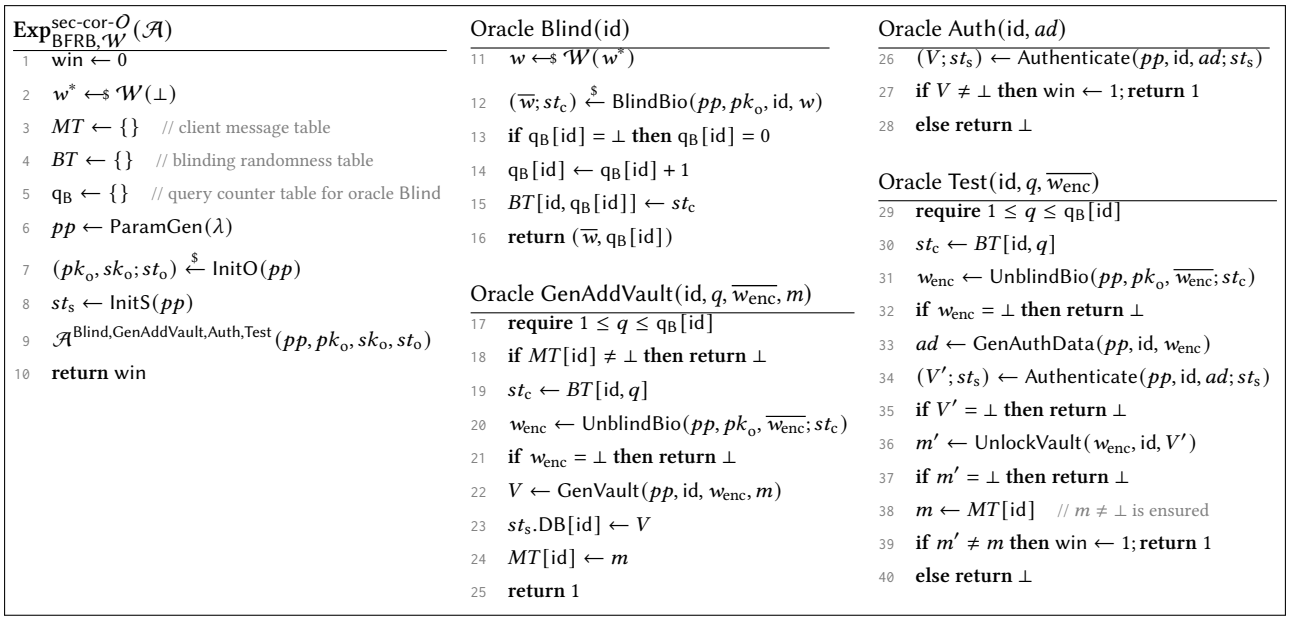

Figure 3: Experiment $\mathbf{Exp}^{\text{sec-cor-}\mathcal{O}}_{\text{BFRB},\mathcal{W}}(\mathcal{A})$ for vault security and client message integrity for corrupted $\mathcal{O}$.

the blinded biometric encoding (line 28). Then the authentication data is created and given to $\mathcal{A}$ (lines 29-31).

ORACLE Test is to capture message integrity when the malicious storage server returns vaults for messages different from what was originally locked. $\mathcal{A}$ has to come up with an id, vault, and authentication data. It "wins" if the vault unlocks to a message different than what was originally locked. We need to do some bookkeeping to record the original messages (lines 33,34). We model

this as part of the indistinguishability experiment as follows. If $b = 0$, then the oracle always returns $\perp$. Otherwise, the oracle returns 1, if the vault unlocks to a different message (lines 36-40). We will require that the adversary cannot distinguish b, which implies that the adversary can never create a vault that unlocks to a different message.

For an adversary $\mathcal{A}$ that actively corrupts the server $\mathcal{S}$, we define its advantage as:

$$\mathsf{Adv}^{\text{sec-cor-}\mathcal{S}}_{\mathsf{BFRB},\mathcal{W}}(\mathcal{A}) = 2\Pr[\mathbf{Exp}^{\text{sec-cor-}\mathcal{S}}_{\mathsf{BFRB},\mathcal{W}}(\mathcal{A}) = 1] - 1 \ .$$

3.2 Security in case of corrupt O

We now present the game-based security definition for the case where the adversary controls the helper server O. The adversary can arbitrarily reply to honest clients' requests during both the Register and Retrieve stages. The adversary wins if it can create any valid authentication data that would allow it to retrieve some vault created by an honest client (if it could, then the attacker could potentially apply a brute force attack and unlock the vault). In addition, the adversary wins if it can make an honest client to retrieve a message different from the one that was originally locked.

The security experiment $\mathbf{Exp}^{\text{sec-cor-}O}_{\mathsf{BFRB},\mathcal{W}}(\mathcal{A})$ is defined in Figure 3. The experiment samples challenge biometric w^* (line 2) and generates public parameters pp and keys for the O server (lines 6,7). Even though server O is malicious, we only consider honestly generated keys. The adversary is given the public parameters and O's public and secret key pair. It is also given access to several oracles.

Oracle Blind lets $\mathcal{A}$ observe the honest clients' (blinded) requests to server O for client ids of its choice. The oracle records the clients' states, which usually contain the blinding randomness (line 15).

Oracle GenAddVault lets the adversary interact with clients as the O server. In addition to selecting the blinded biometric encoding, the adversary can also select the client id and the message that will be hidden in the vault. $\mathcal{A}$ must provide the query index so the experiment can obtain the client's state necessary to unblind the blinded biometric encoding (line 19). We allow one vault per id. The oracle does not return the vault to the attacker because we assume a secure channel between clients and the storage server.

Oracle Auth takes inputs an id and authentication data and tests whether they can fetch a valid vault. In this case, the adversary wins (line 26).

Oracle Test takes inputs an id, blinded biometrics (which can be maliciously created), and the query index. The oracle computes the authentication data that is used to fetch a vault from the storage server (lines 31,32). The adversary wins if the vault opens to a message distinct from the message that was originally locked (lines 34-38).

For an adversary $\mathcal{A}$ that actively corrupts the helper server O, we define its advantage as:

$$\mathsf{Adv}^{\text{sec-cor-}O}_{\mathsf{BFRB},\mathcal{W}}(\mathcal{A}) = \Pr[\mathbf{Exp}^{\text{sec-cor-}O}_{\mathsf{BFRB},\mathcal{W}}(\mathcal{A}) = 1] \ .$$

3.3 Security in case of colluding O and $\mathcal{S}$

The adversary in this case can control both servers. The security experiment is presented in Appendix B.3 in Figure 6. It is a rather straightforward extension of the experiments from Sections 3.1,3.2.

3.4 The case of an authenticated channel

If there is an authenticated channel from O to C, then the security definitions for the corrupted O server and for the case of colluding servers are unaffected. We adapt the security game for the corrupted $\mathcal{S}$ server (Figure 2) accordingly as follows. In the Blind oracle, alongside storing st_c, associated with $(\mathsf{id}, \mathsf{q_B}[\mathsf{id}])$ in the BT table, we also store the blinded value $\overline{w}$. Both the LRVault and GetAuth oracles no longer take $\overline{w_{\mathsf{enc}}}$, as input and $\overline{w_{\mathsf{enc}}}$ used in both oracles is generated using an honest evaluation of OEncBio on $\overline{w_{\mathsf{enc}}}$.

4 Building Blocks

Before we present our BFRB protocol, we define the building blocks and their security.

4.1 Partially Oblivious Pseudorandom Function

An Oblivious Pseudorandom Function (OPRF) [26] is a two-party protocol between a client and a server where the server possessing a secret key obliviously evaluates the PRF output on the client's input without learning that input. A Partially Oblivious Pseudorandom Function (POPRF) [23] is an OPRF variant where the server learns a tag, the public part of the client's input.

POPRF Syntax [38]. A POPRF Fn is an interactive protocol between two parties, a client and the server. It is associated the tag space Fn.T, private input space Fn.X and consists of a tuple of algorithms: (Fn.Setup, Fn.KeyGen, Fn.Req, Fn.BlindEv, Fn.Finalize, Fn.Ev).

- $pp \xleftarrow{\$} \mathsf{Fn.Setup}(\lambda)$, a randomized algorithm that takes as input a security parameter $\lambda \in \mathbb{N}$ and outputs public parameters $pp \in \{0,1\}^*$.
- $(pk, sk) \xleftarrow{\$} \mathsf{Fn.KeyGen}(pp)$, a randomized algorithm run by the server, it takes as input public parameters pp and outputs a public-secret key pair.
- $(req; st) \xleftarrow{\$} \mathsf{Fn.Req}(pp, pk, t, x)$, a randomized algorithm run by the client that takes as input public parameters pp, a public key pk, a public tag $t \in \mathsf{Fn.T}$, and a secret input $x \in \mathsf{Fn.X}$, and outputs a request and a state st.
- $rep \xleftarrow{\$} \mathsf{Fn.BlindEv}(pp, sk, t, req)$, a randomized algorithm run by the server that takes as input public parameters pp, a secret key sk, a public tag t, and a request req, and outputs a reply rep.
- $y \leftarrow \mathsf{Fn.Finalize}(pp, pk, rep; st)$, a deterministic algorithm run by the client that takes as input the public parameters pp, the public key pk, the reply rep, the state st, and outputs a PRF evaluation y or $y = \perp$.
- $y \leftarrow \mathsf{Fn.Ev}(sk, t, x)$, a deterministic algorithm used to define correctness that takes as input the secret key sk, tag $t \in \mathsf{Fn.T}$, input message $x \in \mathsf{Fn.X}$, and outputs a PRF evaluation y.

Correctness. The correctness definition requires that Fn.Ev is a function and that blinded and unblinded evaluations are consistent. Formally, for every security parameter $\lambda \in \mathbb{N}$, every pp output by $\mathsf{Fn.Setup}(1^\lambda)$, every pk, sk output by $\mathsf{Fn.KeyGen}(pp)$, every $t \in \mathsf{Fn.T}$, and every $x \in \mathsf{Fn.X}$, it holds that $\Pr[\mathsf{Fn.Ev}(sk, t, x) = y] = 1$, where the probability is taken over the choice of y as follows: $(req; st) \xleftarrow{\$} \mathsf{Fn.Req}(pp, pk, t, x)$; $rep \xleftarrow{\$} \mathsf{Fn.BlindEv}(pp, sk, t, req)$; $y \leftarrow \mathsf{Fn.Finalize}(pp, pk, rep; st)$.

POPRF Security Definitions.

Verifiability. For the verifiability of the POPRF, we consider both *completeness* and *soundness*. We define these two properties formally using game-based security definitions. For completeness, if the protocol is executed honestly, the function Fn.Finalize should produce the same output as the unblinded Fn.Ev and not result in a decryption error ($\perp$). For soundness, even if an adversary who possesses the secret key deviates from the protocol, it should not be able to forge a reply that causes Fn.Finalize to accept an output inconsistent with the unblinded Fn.Ev. We formalize the completeness and soundness games in Appendix A in Figures 7 and 8, respectively.

The advantage of a Ver-Comp adversary $\mathcal{A}$ against POPRF Fn is defined by

$$\mathsf{Adv}^{\text{VER-COMP}}_{\mathsf{Fn}}(\mathcal{A}) := \Pr\left[\mathbf{Exp}^{\text{VER-COMP}}_{\mathsf{Fn}}(\mathcal{A}) = 1\right] .$$

The advantage of a Ver-Sound adversary $\mathcal{A}$ against POPRF Fn is defined by

$$\mathsf{Adv}^{\text{VER-SOUND}}_{\mathsf{Fn}}(\mathcal{A}) := \Pr\left[\mathbf{Exp}^{\text{VER-SOUND}}_{\mathsf{Fn}}(\mathcal{A}) = 1\right] .$$

Unlinkability. The unlinkablity property of POPRF [38] addresses the input privacy (or request privacy) of the client, assuming that the POPRF server acts as an adversary. There are two unlinkability notions: POPRIV-1, which models the server as a semi-honest adversary, and POPRIV-2, which models it as a malicious adversary. Since POPRIV-1 is sufficient for us, we provide the definition in Figure 9 for reference, writing it in our notation, and not relying on the random oracle, like the definition in [38].

For brevity, we omit explicitly writing the public parameters pp in the algorithm input.

The advantage of a POPRIV-1 adversary $\mathcal{A}$ is defined by

$$\mathsf{Adv}^{\text{popriv1}}_{\mathsf{Fn}}(\mathcal{A}) := \Pr\left[\mathbf{Exp}^{\text{popriv1,1}}_{\mathsf{Fn}}(\mathcal{A}) = 1\right] - \Pr\left[\mathbf{Exp}^{\text{popriv1,0}}_{\mathsf{Fn}}(\mathcal{A}) = 1\right].$$

Strong One-More Unpredictability. We propose a stronger version of the one-more unpredictability, of POPRF in Figure 10, adapted from [23]. The definition requires that no efficient adversary can produce more OPRF outputs than the number of blinded evaluation queries for every public tag.

The advantage of a SOM-Unp adversary $\mathcal{A}$ is defined by

$$\mathsf{Adv}^{\text{SOM-UNP}}_{\mathsf{Fn}}(\mathcal{A}) := \Pr\left[\mathbf{Exp}^{\text{SOM-UNP}}_{\mathsf{Fn}}(\mathcal{A}) = 1\right] .$$

4.2 Authenticated Encryption and PRF

In Appendix A, we define the well-known cryptographic notions of authenticated encryption (AE) and pseudorandom functions (PRF). There we also define their security via $\mathsf{Adv}^{\mathsf{AE}}_{\mathsf{AE}}$, $\mathsf{Adv}^{\mathsf{PRF}}_{F}$, where the former notion captures both goals of message privacy and integrity.

5 BFRB Protocol Construction

Let $\lambda \in \mathbb{N}$ be the security parameter. Let $\mathbb{W}$ be a biometric space, with threshold τ specified on distance metric δ. We assume that $\mathbb{W}$ contains the feature vectors of length fvl extracted from biometric readings, i.e., we assume that feature extractions happen outside of our protocol. Let Fn = (Fn.Setup, Fn.KeyGen, Fn.Req, Fn.BlindEv, Fn.Finalize, Fn.Ev) be a verifiable POPRF with the tag space $\mathsf{Fn.T} = \{0,1\}^*$ and private input space $\mathsf{Fn.X} = \mathbb{W}$. Let AEAD = (Enc, Dec) be an authenticated encryption (AE) scheme with with key space $\mathsf{KeySp} = \{0,1\}^{\mathsf{kl}}$ and message space $\mathsf{MsgSp} = \{0,1\}^*$. Let $F : \{0,1\}^\lambda \times \{0,1\}^n \to \{0,1\}^\lambda$ be a (PRF) function family[5]. Let $\mathsf{H} : \{0,1\}^* \to \{0,1\}^\lambda$ be a hash function.

We present the details of our BFRB protocol in Figures 4,5 and go over the description below.

Setup. In the Setup phase (Figure 4), the public parameters are generated. These include the public parameters for the POPRF (line 1) and the number of subsamples nos and subsample length sl. The parameters also include the maximum counter for rate limiting maxctr (line 2). Finally, the phase generates nos projection indices of length sl (lines 3-6).

All parties learn the public parameters. The $\mathcal{O}$ and $\mathcal{S}$ servers initialize their states. The $\mathcal{O}$ server generates the POPRF keys (line 10). The public key is known to all parties.

Registration. In the Registration phase (Figure 4), the client with id id, as part of the BlindBio algorithm, on input its input biometric vector w and public parameters which include the projection indices, generates nos subsamples w_i of length sl (lines 17-21). The client then blinds each subsample w_i using the POPRFS's Fn.Req algorithm, with $id\|i$ as the public tag (line 22).

The client sends the "blinded" outputs $\overline{w}$ and id to the helper server $\mathcal{O}$ and saves the POPRF's client's state[6] and all w_i (lines 23,24).

The $\mathcal{O}$ server then executes EncBio as follows. If id is new, then the server sets the rate-limiting counter to 0 (line 29). Otherwise, it checks if the requests for id did not exceed the rate limit (via maxctr counter) and if not, runs the POPRF server's Fn.BlindEv procedure on inputs $\overline{w}[i]$ with tag $\mathsf{id}\|i$, and forwards the output $\overline{w_{\mathrm{enc}}}$ to the client. If the rate limit is exceeded, then the server aborts.

The client then executes UnblindBio procedure, for which it runs POPRF's Fn.Finalize on each component of $\overline{w_{\mathrm{enc}}}$ and the saved state (line 40). For our proof to go through, we need to apply a hash to the result (line 41) and to model this hash as a random oracle[7]. The output is vector w_{enc}. The client does not need to store its state after this.

To generate the vault via GenVault procedure, the client uses each component $w_{\mathrm{enc}}[i]$ as the key for the PRF function F to compute a key $vkey_i$ for the authenticated encryption scheme (line 46) and also the authentication data vid (line 45). Then each $V[i]$ has two parts: $V[i].vval$, which is encryption of the client's message m under $vkey_i$, and $V[i].vid$, which is the authenticated data mentioned above. Finally, the client sends the vault V and id to the storage server $\mathcal{S}$.

The storage server runs AddVault by checking if id is new, and then storing V for id id. The server also sets the rate-limiting counter for id to 0 (line 53).

Retrieval. During the Retrieval phase (Figure 5), the client has a new biometric input, and together with the helper server executes the same UnblindBio, EncBio, UnblindBio algorithms as in the Register phase.

[5] In general, the key sizes for the AE and PRF do not have to be the same, but in practice they are often AES keys so we keep them the same for simplicity.

[6] This usually is the randomness used to blind and unblind the input.

[7] Although we state it explicitly, this hash is a part of the 3HashSDHI POPRF, and hence does not impact the efficiency of our construction.

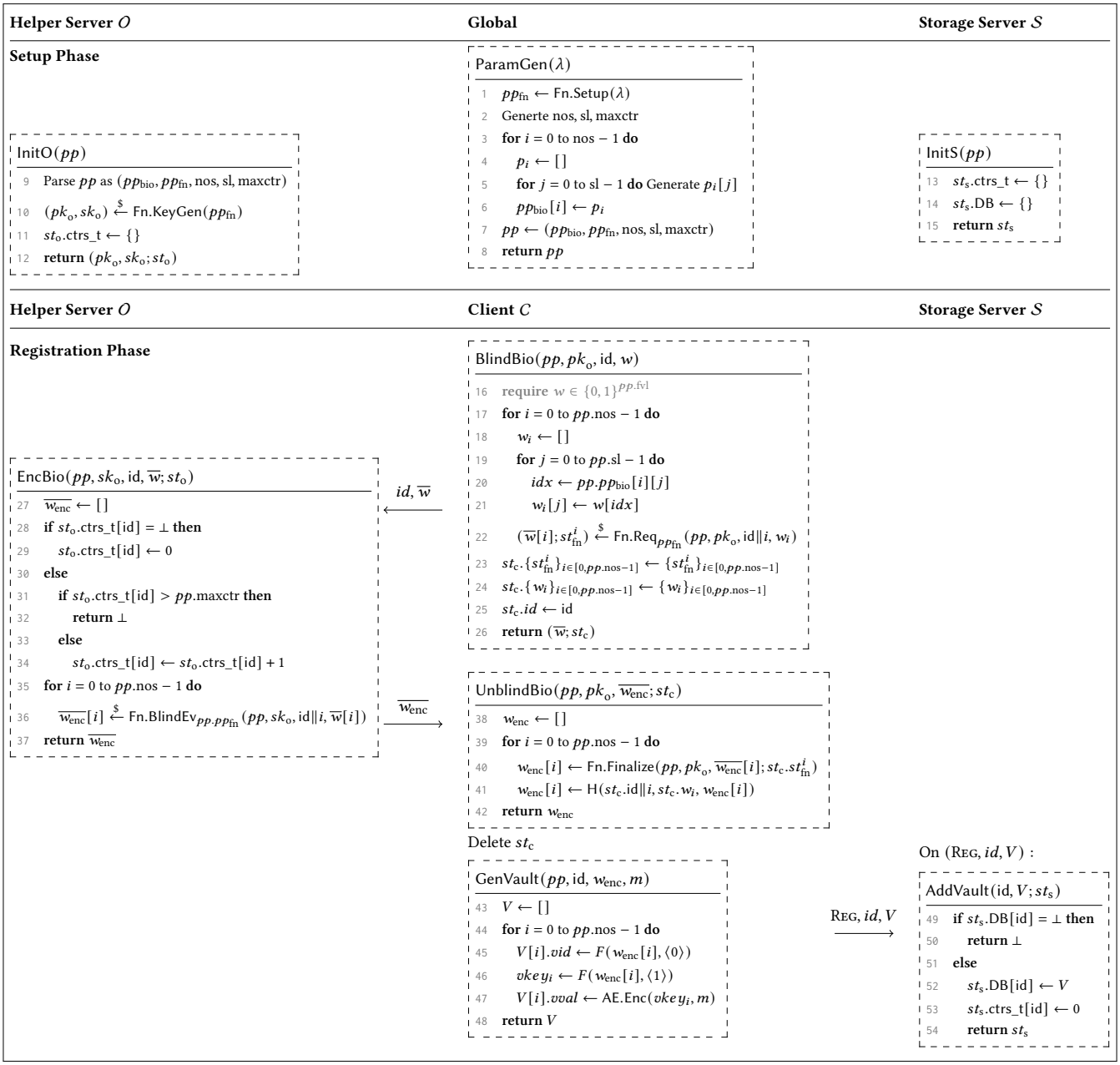

Figure 4: Setup and Registration phases of BFRB protocol.

Next, using the resulting w'_{enc}, the client computes the authentication data ad using the PRF F on fixed input under each $w'_{\text{enc}}[i]$ as the key (line 84). Those are sent to the storage server along with the client's id id. The storage server checks that there is a vault stored for id, and also checks if, for any vault's component i, the authentication data matches: $V[i].vid = ad[i]$. If there is a match, then the corresponding vault and index i are returned to the client. The storage server also maintains the counter for rate-limiting and checks that it is not exceeded.

Finally, the client runs UnlockVault as follows. The client computes the decryption key using F and $w'_{\text{enc}}[i]$ as the key (line 101), and decrypts $V[i].vval$ to obtain message m.

Correctness. Assuming the correctness of the POPRF and AE, the correctness of our scheme follows directly from the correctness of the underlying sample-then-lock [11] protocol. We use the IrisLock [2] protocol in our instantiation, which achieves varying levels of correctness error based on the parameters chosen for

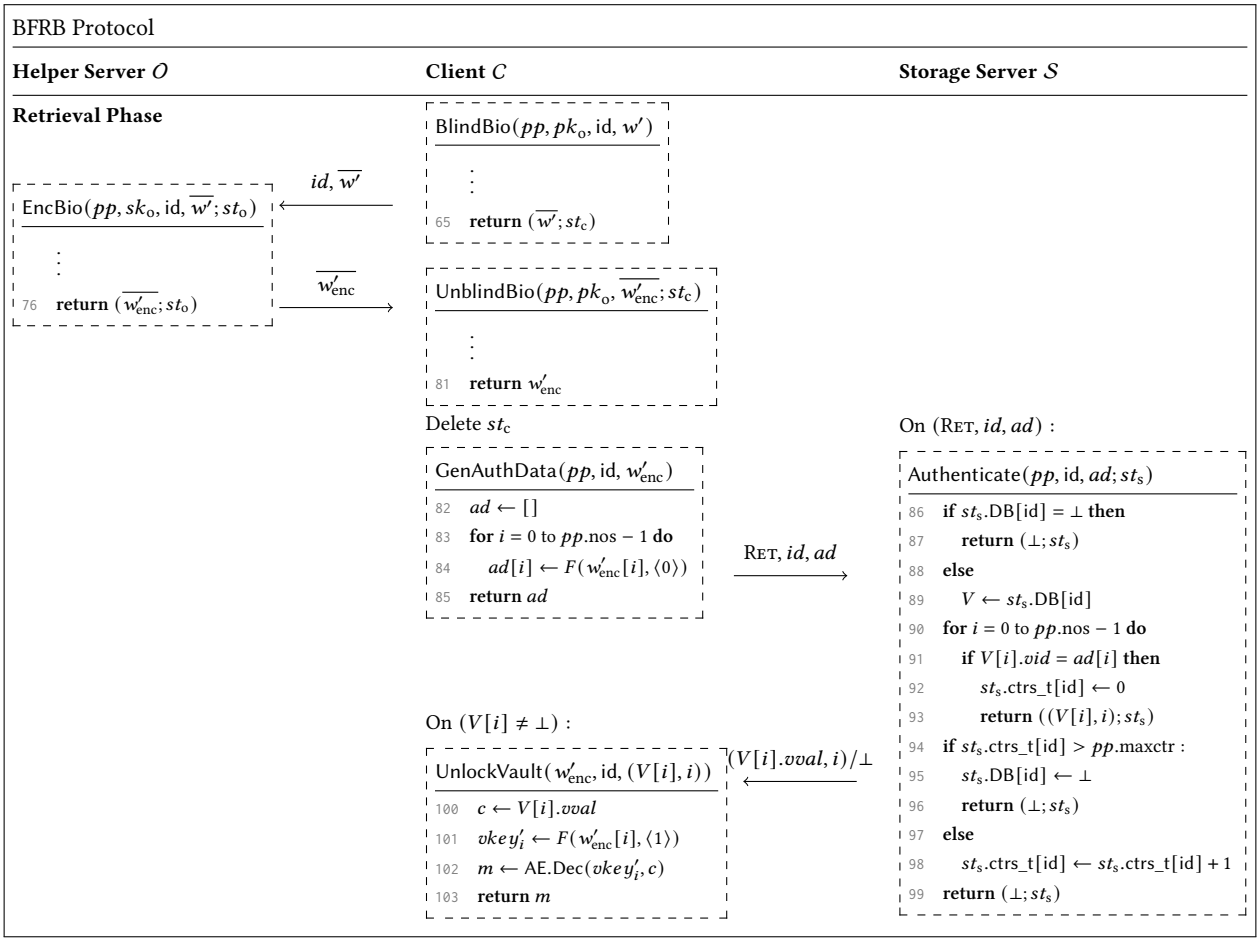

Figure 5: Retrieval phase of BFRB protocol.

their ζ-sampling algorithm [2, Tables 2,3]. For our recommended parameters, BFRB achieves a best-case correctness error $\chi = 0.13$.

6 BFRB Security Analysis

We state the security guarantees our BFRB protocol (defined in the previous section) provides. In following security statements, we assume the min-entropy of biometric subsamples of BFRB is ρ.

Security in case of corrupt $\mathcal{S}$. We show that in case of corrupted $\mathcal{S}$ server, our BFRB protocol is secure, according to the definition in Section 3.1, assuming the underlying PRF and authenticated encryption are secure, and the OPRF is verifiable and provides one-more unpredictability and semi-honest unlinkability, and when the hash function is modeled as a random oracle.

Theorem 1. *Let $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$ be an efficient adversary that makes at most $(q_B, q_E, q_{LR}, q_{auth}, q_{test})$ queries per* id *to its respective* Blind, OEncBio, LRVault, GetAuth, Test *oracles. Then we construct efficient adversaries $\mathcal{A}_{\text{som-unp}}$, $\mathcal{A}_{\text{ver-sound}}$, $\mathcal{A}_{\text{popriv1}}$, $\mathcal{A}_{\text{prf}}$, $\mathcal{A}_{\text{ae}}$ such that*

$$\begin{aligned}
&\mathsf{Adv}^{\text{sec-cor-}\mathcal{S}}_{\text{BFRB},\mathcal{W}}(\mathcal{A}_{\text{sec-cor-}\mathcal{S}}) \\
&\leq 2pp.\text{nos} \cdot (q_{LR} + q_{auth}) \cdot \mathsf{Adv}^{\text{VER-SOUND}}_{\mathsf{Fn}}(\mathcal{A}_{\text{ver-sound}}) \\
&+ 2\mathsf{Adv}^{\text{SOM-UNP}}_{\mathsf{Fn}}(\mathcal{A}_{\text{som-unp}}) \\
&+ 2pp.\text{nos} \cdot q_B \cdot \mathsf{Adv}^{\text{popriv1}}_{\mathsf{Fn}}(\mathcal{A}_{\text{popriv1}}) \\
&+ \frac{pp.\text{nos} \cdot (q_E - 1)}{2^{\rho-1}} + 2pp.\text{nos} \cdot (q_{LR} + q_{auth}) \cdot \mathsf{Adv}^{\text{PRF}}_{F}(\mathcal{A}_{\text{prf}}) \\
&+ 4pp.\text{nos} \cdot q_{LR} \cdot \mathsf{Adv}^{\text{AE}}_{\text{AE}}(\mathcal{A}_{\text{ae}}) \ .
\end{aligned}$$

The detailed proof is in Appendix B.1. See Section 6.1 for the explanations of the effect of rate limiting on security.

Security in case of corrupt O. We show that in case of corrupted O server, our BFRB protocol is secure, according to the definition in Section 3.2, assuming the underlying PRF and authenticated encryption are secure, and the OPRF is verifiable and provides semi-honest unlinkability, and when the hash function is modeled as a random oracle.

Theorem 2. *Let $\mathcal{A}_{\text{sec-cor-}O}$ be an efficient adversary that makes at most $(q_B, q_{vault}, q_{auth}, q_{test})$ queries per* id *to its respective* Blind,

GenAddVault, Auth, Test *oracles. We then construct efficient adversary* $\mathcal{A}_{\text{ver-sound}}$*, unlinkability (*POPRIV-1*) adversary* $\mathcal{A}_{\text{popriv1}}$*, PRF adversary* $\mathcal{A}_{\text{prf}}$*, and AE adversary* $\mathcal{A}_{\text{ae}}$ *such that*

$$\begin{aligned}
&\mathsf{Adv}^{\text{sec-cor-}\mathcal{O}}_{\text{BFRB},\mathcal{W}}(\mathcal{A}_{\text{sec-cor-}\mathcal{O}}) \\
&\leq 2pp.\text{nos} \cdot (q_{\text{vault}} + q_{\text{test}}) \cdot \mathsf{Adv}^{\text{VER-SOUND}}_{\mathsf{Fn}}(\mathcal{A}_{\text{ver-sound}}) \\
&+ 2pp.\text{nos} \cdot q_{\mathsf{B}} \cdot \mathsf{Adv}^{\text{popriv1}}_{\mathsf{Fn}}(\mathcal{A}_{\text{popriv1}}) + \frac{pp.\text{nos} \cdot q_{\text{auth}}}{2^{\rho-1}} \\
&+ 2pp.\text{nos} \cdot (q_{\text{vault}} + q_{\text{test}}) \cdot \mathsf{Adv}^{\text{PRF}}_{F}(\mathcal{A}_{\text{prf}}) \\
&+ 4pp.\text{nos} \cdot q_{\text{vault}} \cdot \mathsf{Adv}^{\text{AE}}_{\mathsf{AE}}(\mathcal{A}_{\text{ae}}) \ .
\end{aligned}$$

The detailed proof is in Appendix B.2. See Section 6.1 for the explanations of the effect of rate limiting on security.

Security in case of corrupt $\mathcal{S}$ and $\mathcal{O}$. If both servers are corrupted and can collude, we cannot prevent the adversary from brute-forcing the user biometric. But we show that any attack has to brute-force.

Biometric Brute-forcing. We first define brute-forcing the biometric, similarly to how it is defined for passwords in [5].

For parameter pp, biometric sampler $\mathcal{W}$, and adversary $\mathcal{A}$, let $\mathbf{Exp}^{\text{bio-guess}}_{pp,\mathcal{W}}(\mathcal{A})$ be the biometric guessing game as defined in Figure 11 in Appendix. We define the biometric-guessing advantage of an adversary as:

$$\mathsf{Adv}^{\text{bio-guess}}_{pp,\mathcal{W}}(\mathcal{A}) = \Pr[\mathbf{Exp}^{\text{bio-guess}}_{pp,\mathcal{W}}(\mathcal{A})].$$

Theorem 3 in Appendix B.3 shows that for an adversary $\mathcal{A}_{\text{sec-cor-}\mathcal{S}\text{-}\mathcal{O}}$ defined in Section 3.3, there exists an adversary $\mathcal{A}$, such that $\mathsf{Adv}^{\text{sec-cor-}\mathcal{S}\text{-}\mathcal{O}}_{\text{BFRB},\mathcal{W}}(\mathcal{A}_{\text{sec-cor-}\mathcal{S}\text{-}\mathcal{O}})$ is upper-bounded by $\mathsf{Adv}^{\text{bio-guess}}_{pp,\mathcal{W}}(\mathcal{A})$, and other terms related to the other building blocks. This shows that even if both servers are corrupted, the attacker has to brute-force the user's biometric (or break other building blocks).

Security in case of the authenticated channel. If there is an authenticated channel between $\mathcal{O}$ server and the client, then security holds without using the verifiability of POPRF during the Retrieval phase, yielding efficiency improvements, assessed in Section 8. We do require an additional assumption for the authenticated encryption. The detailed security analysis is in Appendix B.4.

6.1 Rate Limiting

We illustrate the effectiveness of rate limiting by showing that adversarial advantage in our security games remains small. In more detail, Theorem 1 quantifies the advantage of the adversary with respect to the min-entropy of the subsamples and the number oracle calls required. This advantage is effectively bounded in a certain period of time using the rate-limiting strategy.

Recall that we enforce rate limiting on both the OPRF and storage servers. As shown in Figures 4 and 5, both servers keep per-user-id rate-limiting counters (no signature required), ctrs_t, in a map format for multiple clients. For simplicity, we assume that this counter set, ctrs_t, automatically resets to 0 for each client every hour. Similar to the Pythia service [24], we set the maximum counter maxctr per hour to 10 and cap the monthly limit at 300. Choosing projection parameters for a low correctness error $\chi = 0.13$ that achieve $\rho = 33$ with $pp.\text{nos} = 200K$ the term $\frac{pp.\text{nos}\cdot(q_{\mathsf{E}}-1)}{2^{\rho-1}}$ (Theorem 1) approximately equals 1 when $q_{\mathsf{E}} = 21475$. This means that, with rate limiting, it would take roughly 5.97 years to achieve an advantage of 1, assuming the remaining advantage terms are negligible. Choosing projection parameters with a higher correctness error $\chi = 0.45$ that achieve $\rho = 42$ with $pp.\text{nos} = 200K$ the same term $\frac{pp.\text{nos}\cdot(q_{\mathsf{E}}-1)}{2^{\rho-1}}$ (Theorem 1) approximately equals 1 when $q_{\mathsf{E}} = 10995117$, which would take roughly 3054 years to recover. As the `IrisLock` ζ-sampling algorithm offers a variety of parameter choices that tradeoff min-entropy vs. correctness error [2, Table 3], users that deploy the BFRB protocol may choose parameters from between these two extremes as per their security considerations[8].

We note that similarly to password-based authentication, rate-limiting introduces a risk of Denial-of-Service attacks. This can be problematic in certain scenarios where users might be unable to log in because some adversary has already exhausted the login budget. This attack can be mitigated using usual measures such as CAPTCHAs. In addition, the users might be unable to log in if the the True Accept Rate (TAR) is low and the rate-limiting threshold is set too low. However, if the threshold is set too high, this may give adversaries enough attempts to recover the biometric subsamples. Thus, it is essential to set appropriate rate-limiting thresholds that balance TAR and the security requirements suitable for specific applications.

7 Concrete Instantiation

Feature Extraction & Projection Sampling. We use the biometric feature extractors and the subsample generation procedure as described in the `IrisLock` [2] protocol for biometric key derivation. The `IrisLock` protocol defines a CNN-based heterogeneous feature extractor to extract 1024-bit features from segmented iris images from the ND-0405 dataset [9] (where the images are segmented using [3]). The rest of their protocol is a slightly modified variant of the sample-then-lock fuzzy extractor construction [11]. Instead of sampling uniformly random projections as in [11], the `IrisLock` protocol uses a proprietary ζ-sampling algorithm to fix optimal projection indices. Using these pre-determined projections, `IrisLock` subsamples optimize the tradeoff between entropy and the True Accept Rate at the cost of reusability.

While their min-entropy estimates are state-of-the-art, with their "optimal" projection parameters generating subsamples with a min-entropy as high as 42 bits, due to the underlying limitations of sample-then-lock, the TAR levels at which this entropy is possible is miniscule. `IrisLock` claims a TAR value of 0.12 (i.e., correctness error $\chi = 0.88$), which can be boosted to a 0.31 by grouping together[9] 5 iris readings, and upto a maximum of 0.45 with 21 readings [2, Table 4]. For `IrisLock`, this correctness error is unavoidable as they need their min-entropy readings to be high enough for their system to offer similar security as using traditional passwords. However, due to the protections in place against brute-force adversaries, the BFRB protocol offers better security even assuming a lower subsample min-entropy. This allows us to select projection parameters that

[8]The choice of projection parameters does not impact the performance of our protocol as long as the number of subsamples remains constant.

[9]Extracting features from multiple different readings (i.e., images) of the same iris and generating a combined feature vector by using the majority bit at each index

	BlindBio	EncBio (Retrieve)	UnblindBio (Retrieve)	Total (Register)	Register + net. delay 100 Mbps	Register + net. delay 1 Gbps	Total (Retrieve)	Retrieve + net. delay 100 Mbps	Retrieve + net. delay 1 Gbps
BFRB	5686	12011	16338	34469	41209	35278	34172	40004	34935
BFRB$^+$		6651	6857				19331	25163	20094
BFRB MT	803	1487	2213	4876	11616	5685	4640	10472	5403
BFRB$^+$ MT		760	884				2584	8416	3347

Table 1: Performance analysis of protocol steps. All times measured in milliseconds. MT - Multi-threaded performance with 20 concurrent threads. BFRB$^+$ is the protocol assuming authenticated channel to server O. Times for BlindBio, EncBio**, and** UnblindBio **for BFRB are the same during Register and Retrieve. For BFRB$^+$, these times for the Register phase are the same as for BFRB.**

achieve a higher TAR value at the cost of min-entropy. We generate $pp.\text{nos} = 200,000$ projections of length $k = 60$ bits each using the `UnlikeExp` algorithm with $\zeta = 0.5$. Using these projections, our generated subsamples have a min-entropy of 33 bits while achieving a correctness error $\chi = 0.59$ (TAR = 0.41). This error is further reduced to $\chi = 0.24$ (TAR = 0.76) by grouping 5 iris readings, up to a minimum of $\chi = 0.13$ (TAR = 0.87) with 21 readings [2, Table 4].

VPOPRF. We use the 3HashSDHI construction [38], which is the state-of-the-art VPOPRF protocol both in terms of performance and security. 3HashSDHI modifies the widely known 2HashDH OPRF protocol to allow for public tags and uses NIZK proofs for verifiability. We instantiate 3HashSDHI using the ristretto255 group [18] and the SHA-512 hash function. 3HashSDHI has been shown to satisfy PRF security under a malicious client and unlinkability under malicious and semi-honest servers, relying on the one-more gap strong Diffie-Hellman inversion assumption in the random oracle model. It remains to show that 3HashSDHI satisfies one-more unpredictability. We address this in Appendix B.5. The proof relies on the one-more gap strong Diffie-Hellman inversion assumption [38] in the random oracle model.

Other building block instantiations. We use AES-256 as a PRF and AES-256 with the GCM-SIV mode of operation [29] as authenticated encryption. AES-GCM-SIV satisfies both confidentiality (IND-CPA) and ciphertext integrity (INT-CTXT) notions while also having misuse resistance in case of nonce reuse.

8 Performance

To analyze the efficiency of the BFRB protocol, we implement the protocol in Rust. We use Facebook's Rust `voprf` crate implementing the 3HashSDHI protocol[10] and the RustCrypto implementation of AES-GCM-SIV[11]. The `voprf` crate also implements batching for 3HashSDHI, i.e., generating a single NiZK proof for multiple ORPF evaluations, which further improves efficiency. All timings are collected on a machine with an Intel Core i7 12700H CPU and 24 gigabytes of RAM, running Ubuntu 24.04. The timing figures reported are mean values across 100 runs. We further estimate a network overhead by assuming 100 Mbps and 1 Gbps bandwidths (modeling cellular and wired networks respectively) and a network latency of 50 milliseconds (≥ approx. latency to AWS servers in the North America region [15]).

The efficiency analysis of our protocol, with a detailed analysis of each VPOPRF phase, can be found in Figure 1. Total time for Register also includes the time for GenVault (434 ms). Total time for Retrieve also includes the time for GenAuthData (128 ms), Authenticate (9 ms), and UnlockVault (< 1ms)[12]. Our `Retrieve` phase using verifiable POPRF takes < 5.5 seconds including approximate network overhead in a gigabit network while executing on a consumer-grade laptop processor. Our implementation is available at https://github.com/deep-inder/BFRB.

[10] https://github.com/facebook/voprf

[11] https://github.com/RustCrypto/AEADs

[12] Execution times for GenVault, GenAuthData, Authenticate, and UnlockVault are similar across single and multi-threaded execution.

Acknowledgments

The authors thank Benjamin Fuller, Amey Shukla, and the rest of the team behind `IrisLock` [2] for their many insights into their protocol and its implementation, and for providing us a pretrained iris feature extractor. The authors thank Joseph Jaeger, Andreas Hülsing, and Rei Safavi-Naini for their helpful comments and discussion. The authors are also grateful to the ACM CCS '25 reviewers for their helpful comments. Alexandra Boldyreva and Deep Inder Mohan were supported by the National Science Foundation under Grant No.1946919. Tianxin Tang is supported by an NWO VIDI grant (Project No. VI.Vidi.193.066).

References

[1] Shashank Agrawal, Saikrishna Badrinarayanan, Pratyay Mukherjee, and Peter Rindal. 2020. Game-Set-MATCH: Using Mobile Devices for Seamless External-Facing Biometric Matching. In *ACM CCS 2020*, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM Press, 1351–1370. https://doi.org/10.1145/3372297.3417287

[2] Sohaib Ahmad, Sixia Chen, Luke Demarest, Benjamin Fuller, Caleb Manicke, Alexander Russell, and Amey Shukla. 2024. IrisLock: Iris Biometric Key Derivation with 42 bits of security. Cryptology ePrint Archive, Paper 2024/100. https://eprint.iacr.org/archive/2024/100/20241106:010157

[3] Sohaib Ahmad and Benjamin Fuller. 2019. Unconstrained Iris Segmentation Using Convolutional Neural Networks. In *Computer Vision – ACCV 2018 Workshops*, Gustavo Carneiro and Shaodi You (Eds.). Springer International Publishing, Cham, 450–466.

[4] Michael Armbrust, Armando Fox, Rean Griffith, Anthony D. Joseph, Randy H. Katz, Andrew Konwinski, Gunho Lee, David A. Patterson, Ariel Rabkin, and Matei Zaharia. 2009. *Above the Clouds: A Berkeley View of Cloud Computing*. Technical Report.

[5] Matilda Backendal, Hannah Davis, Felix Günther, Miro Haller, and Kenneth G. Paterson. 2024. A Formal Treatment of End-to-End Encrypted Cloud Storage. In *CRYPTO 2024, Part II (LNCS, Vol. 14921)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, Cham, 40–74. https://doi.org/10.1007/978-3-031-68379-4_2

[6] Pia Bauspiess, Tjerand Silde, Matej Poljuha, Alexandre Tullot, Anamaria Costache, Christian Rathgeb, Jascha Kolberg, and Christoph Busch. 2024. BRAKE: Biometric Resilient Authenticated Key Exchange. *IEEE Access* 12 (2024), 46596–46615. https://doi.org/10.1109/ACCESS.2024.3380915

[7] Mihir Bellare and Phillip Rogaway. 2006. The Security of Triple Encryption and a Framework for Code-Based Game-Playing Proofs. In *EUROCRYPT 2006*

(LNCS, Vol. 4004), Serge Vaudenay (Ed.). Springer, Berlin, Heidelberg, 409–426. https://doi.org/10.1007/11761679_25

[8] Marina Blanton and Dennis Murphy. 2024. Privacy Preserving Biometric Authentication for Fingerprints and Beyond. In *Proceedings of the Fourteenth ACM Conference on Data and Application Security and Privacy* (Porto, Portugal) *(CODASPY '24)*. Association for Computing Machinery, New York, NY, USA, 367–378. https://doi.org/10.1145/3626232.3653269

[9] Kevin W. Bowyer and Patrick J. Flynn. 2016. The ND-IRIS-0405 Iris Image Dataset. *CoRR* abs/1606.04853 (2016). arXiv:1606.04853 http://arxiv.org/abs/1606.04853

[10] Xavier Boyen, Yevgeniy Dodis, Jonathan Katz, Rafail Ostrovsky, and Adam Smith. 2005. Secure Remote Authentication Using Biometric Data. In *Advances in Cryptology – EUROCRYPT 2005*, Ronald Cramer (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 147–163.

[11] Ran Canetti, Benjamin Fuller, Omer Paneth, Leonid Reyzin, and Adam D. Smith. 2016. Reusable Fuzzy Extractors for Low-Entropy Distributions. In *EUROCRYPT 2016, Part I (LNCS, Vol. 9665)*, Marc Fischlin and Jean-Sébastien Coron (Eds.). Springer, Berlin, Heidelberg, 117–146. https://doi.org/10.1007/978-3-662-49890-3_5

[12] Sílvia Casacuberta, Julia Hesse, and Anja Lehmann. 2022. SoK: Oblivious Pseudorandom Functions. Cryptology ePrint Archive, Report 2022/302. https://eprint.iacr.org/2022/302

[13] Yu Chen and Yiling He. 2023. BrutePrint: Expose Smartphone Fingerprint Authentication to Brute-force Attack. arXiv:2305.10791 [cs.CR] https://arxiv.org/abs/2305.10791

[14] Yu Chen, Yang Yu, and Lidong Zhai. 2023. InfinityGauntlet: Expose Smartphone Fingerprint Authentication to Brute-force Attack. In *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023*, Joseph A. Calandrino and Carmela Troncoso (Eds.). USENIX Association, 2027–2041. https://www.usenix.org/conference/usenixsecurity23/presentation/chen-yu

[15] The Khang Dang, Nitinder Mohan, Lorenzo Corneo, Aleksandr Zavodovski, Jörg Ott, and Jussi Kangasharju. 2021. Cloudy with a chance of short RTTs: analyzing cloud connectivity in the internet. In *Proceedings of the 21st ACM Internet Measurement Conference* (Virtual Event) *(IMC '21)*. Association for Computing Machinery, New York, NY, USA, 62–79. https://doi.org/10.1145/3487552.3487854

[16] Gareth T. Davies, Sebastian Faller, Kai Gellert, Tobias Handirk, Julia Hesse, Máté Horváth, and Tibor Jager. 2023. Security Analysis of WhatsApp End-to-End Encrypted Backup Protocol. In *Advances in Cryptology – CRYPTO 2023: 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20–24, 2023, Proceedings, Part IV* (Santa Barbara, CA, USA). Springer-Verlag, Berlin, Heidelberg, 330–361. https://doi.org/10.1007/978-3-031-38551-3_11

[17] Ivan De Oliveira Nunes, Peter Rindal, and Maliheh Shirvanian. 2024. Oblivious Extractors and Improved Security in Biometric-Based Authentication Systems. In *Computer Security – ESORICS 2023*, Gene Tsudik, Mauro Conti, Kaitai Liang, and Georgios Smaragdakis (Eds.). Springer Nature Switzerland, Cham, 290–312.

[18] Henry de Valence, Jack Grigg, Mike Hamburg, Isis Lovecruft, George Tankersley, and Filippo Valsorda. 2023. The ristretto255 and decaf448 Groups. RFC 9496. https://doi.org/10.17487/RFC9496

[19] Yevgeniy Dodis, Leonid Reyzin, and Adam Smith. 2004. Fuzzy Extractors: How to Generate Strong Keys from Biometrics and Other Noisy Data. In *EUROCRYPT 2004 (LNCS, Vol. 3027)*, Christian Cachin and Jan Camenisch (Eds.). Springer, Berlin, Heidelberg, 523–540. https://doi.org/10.1007/978-3-540-24676-3_31

[20] Pierre-Alain Dupont, Julia Hesse, David Pointcheval, Leonid Reyzin, and Sophia Yakoubov. 2018. Fuzzy Password-Authenticated Key Exchange. In *Advances in Cryptology – EUROCRYPT 2018*, Jesper Buus Nielsen and Vincent Rijmen (Eds.). Springer International Publishing, Cham, 393–424.

[21] Joshua J. Engelsma, Kai Cao, and Anil K. Jain. 2021. Learning a Fixed-Length Fingerprint Representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43, 6 (2021), 1981–1997. https://doi.org/10.1109/TPAMI.2019.2961349

[22] Andreas Erwig, Julia Hesse, Maximilian Orlt, and Siavash Riahi. 2020. Fuzzy Asymmetric Password-Authenticated Key Exchange. In *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 12492)*, Shiho Moriai and Huaxiong Wang (Eds.). Springer, 761–784. https://doi.org/10.1007/978-3-030-64834-3_26

[23] Adam Everspaugh, Rahul Chaterjee, Samuel Scott, Ari Juels, and Thomas Ristenpart. 2015. The Pythia PRF Service. In *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, Washington, D.C., 547–562. https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/everspaugh

[24] Adam Everspaugh, Rahul Chatterjee, Samuel Scott, Ari Juels, and Thomas Ristenpart. 2015. The Pythia PRF Service. In *USENIX Security 2015*, Jaeyeon Jung and Thorsten Holz (Eds.). USENIX Association, 547–562.

[25] Sebastian Faller, Tobias Handirk, Julia Hesse, Máté Horváth, and Anja Lehmann. 2024. Password-Protected Key Retrieval with(out) HSM Protection. ACM CCS. https://doi.org/10.1145/3658644.3690358

[26] Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold. 2005. Keyword Search and Oblivious Pseudorandom Functions. In *TCC 2005 (LNCS, Vol. 3378)*, Joe Kilian (Ed.). Springer, Berlin, Heidelberg, 303–324. https://doi.org/10.1007/978-3-540-30576-7_17

[27] GDPR.EU. 2020. GDPR compliance. https://gdpr.eu/.

[28] Maksymilian Gorski and Wojciech Wodo. 2024. Analysis of Biometric-Based Cryptographic Key Exchange Protocols—BAKE and BRAKE. *Cryptography* 8, 2 (2024). https://doi.org/10.3390/cryptography8020014

[29] Shay Gueron, Adam Langley, and Yehuda Lindell. 2019. AES-GCM-SIV: Nonce Misuse-Resistant Authenticated Encryption. RFC 8452. https://doi.org/10.17487/RFC8452

[30] Horizon.Research. 2024. Horizon Grand View Research. Global Biometric Technology Market Size & Outlook. https://www.grandviewresearch.com/horizon/outlook/biometric-technology-market-size/global.

[31] ISO.ORG. 2022. ISO/IEC 24745:2022 Information security, cybersecurity and privacy protection — Biometric information protection. https://www.iso.org/standard/75302.html.

[32] Stanislaw Jarecki, Hugo Krawczyk, Maliheh Shirvanian, and Nitesh Saxena. 2016. Device-Enhanced Password Protocols with Optimal Online-Offline Protection. In *Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security* (Xi'an, China) *(ASIA CCS '16)*. Association for Computing Machinery, New York, NY, USA, 177–188. https://doi.org/10.1145/2897845.2897880

[33] Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. 2018. OPAQUE: An Asymmetric PAKE Protocol Secure Against Pre-computation Attacks. In *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part III (Lecture Notes in Computer Science, Vol. 10822)*, Jesper Buus Nielsen and Vincent Rijmen (Eds.). Springer, 456–486. https://doi.org/10.1007/978-3-319-78372-7_15

[34] Ari Juels and Madhu Sudan. 2006. A Fuzzy Vault Scheme. *DCC* 38, 2 (2006), 237–257. https://doi.org/10.1007/s10623-005-6343-z

[35] Maliheh Shirvanian, Christopher Robert Price, Mohammed Jubur, Nitesh Saxena, Stanislaw Jarecki, and Hugo Krawczyk. 2021. A hidden-password online password manager. In *Proceedings of the 36th Annual ACM Symposium on Applied Computing* (Virtual Event, Republic of Korea) *(SAC '21)*. Association for Computing Machinery, New York, NY, USA, 1683–1686. https://doi.org/10.1145/3412841.3442131

[36] Sailesh Simhadri, James Steel, and Benjamin Fuller. 2019. Cryptographic Authentication from the Iris. In *Information Security*, Zhiqiang Lin, Charalampos Papamanthou, and Michalis Polychronakis (Eds.). Springer International Publishing, Cham, 465–485.

[37] Nirvan Tyagi, Sofía Celi, Thomas Ristenpart, Nick Sullivan, Stefano Tessaro, and Christopher A. Wood. 2022. A Fast and Simple Partially Oblivious PRF, with Applications. In *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 13276)*, Orr Dunkelman and Stefan Dziembowski (Eds.). Springer, 674–705. https://doi.org/10.1007/978-3-031-07085-3_23

[38] Nirvan Tyagi, Sofía Celi, Thomas Ristenpart, Nick Sullivan, Stefano Tessaro, and Christopher A. Wood. 2022. A Fast and Simple Partially Oblivious PRF, with Applications. In *EUROCRYPT 2022, Part II (LNCS, Vol. 13276)*, Orr Dunkelman and Stefan Dziembowski (Eds.). Springer, Cham, 674–705. https://doi.org/10.1007/978-3-031-07085-3_23

[39] Mei Wang, Kun He, Jing Chen, Zengpeng Li, Wei Zhao, and Ruiying Du. 2021. Biometrics-Authenticated Key Exchange for Secure Messaging. In *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, Yongdae Kim, Jong Kim, Giovanni Vigna, and Elaine Shi (Eds.). ACM, 2618–2631. https://doi.org/10.1145/3460120.3484746

[40] Matthew R. Young, Stephen J. Elliott, Catherine J. Tilton, and James E. Goldman. 2013. *International Journal of Computer Science Engineering and Technology (IJCSET)* 3, 2 (2013), 43–47.

A Cryptographic Building Blocks

In this section, we provide for reference the syntax and security properties of the cryptographic building blocks of our protocol: authenticated encryption (AE) and pseudorandom function family (PRF).

A.1 Authenticated Encryption (AE)

Authenticated encryption (AE) is symmetric encryption that provides both *confidentiality* and *authenticity*. AE includes blockcipher-based mode of operations such as AES-GCM. Popular variants are nonce-based and support (authenticated) associated data. Since our

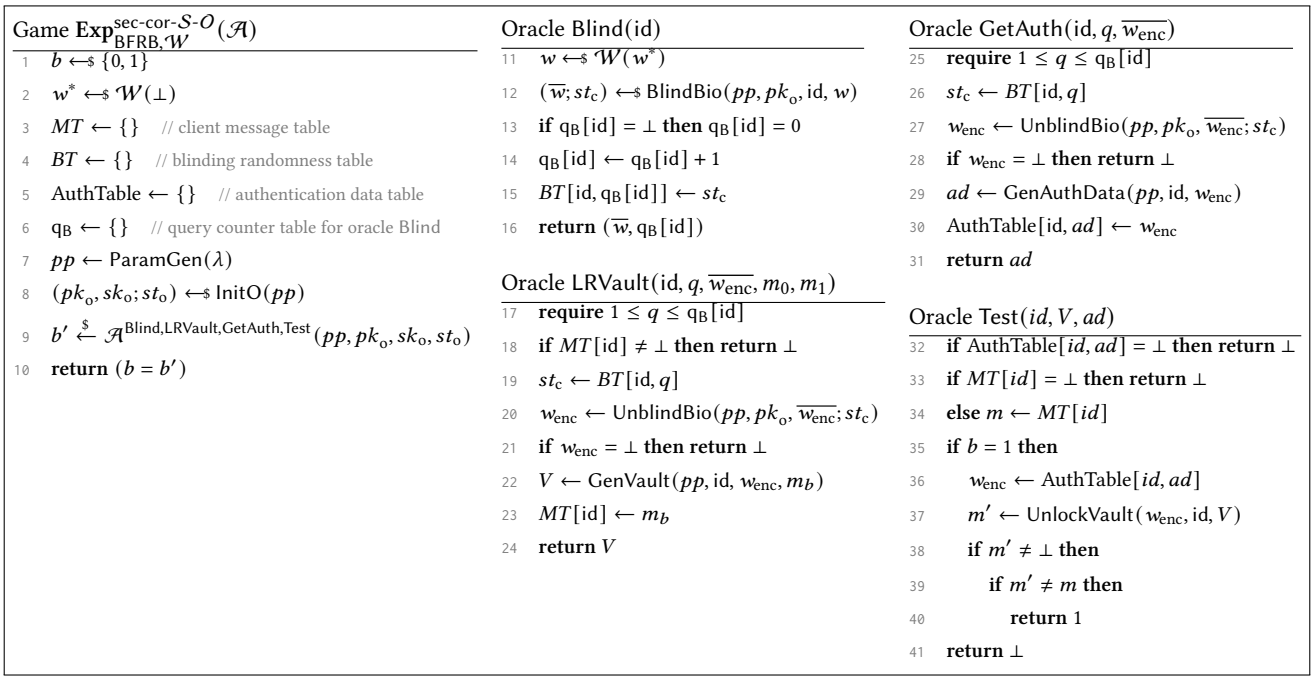

Figure 6: Experiment $\mathbf{Exp}^{\text{sec-cor-}\mathcal{S}\text{-}\mathcal{O}}_{\text{BFRB},\mathcal{W}}(\mathcal{A})$ for message indistinguishability and integrity for corrupted and colluding $\mathcal{S}$ and $\mathcal{O}$.

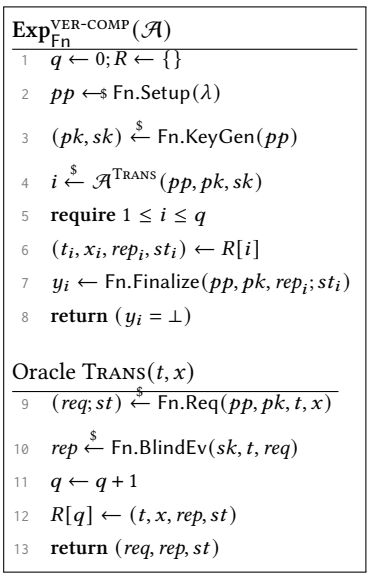

Figure 7: Completeness experiment for verifiable POPRF Fn.

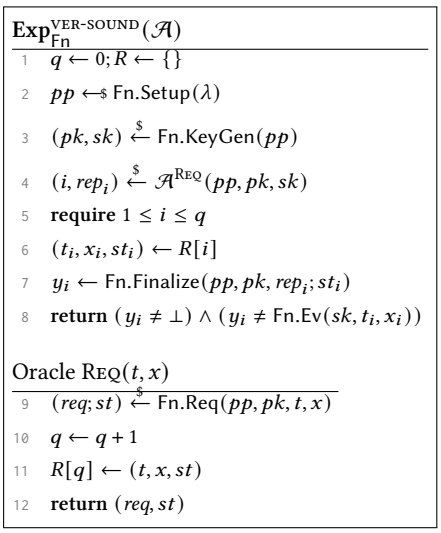

Figure 8: Soundness experiment for verifiable POPRF Fn.

construction does not need associated data, we omit it in the syntax. In addition, for generality, we omit the nonce in the syntax. AE provides confidentiality (IND$-CPA) and ciphertext integrity (INT-CTXT). We provide the all-in-one notion of authenticated encryption here.

Syntax. AE contains the following algorithms,

- $c \xleftarrow{\$}$ AE.Enc(sk, m): a randomized algorithm that takes as input a secret key $sk \in \{0,1\}^{\mathrm{kl}}$ for some key length kl, a message $m \in \{0,1\}^*$, and outputs a ciphertext c.
- $m \leftarrow$ AE.Dec(sk, c): a deterministic algorithm that takes as input secret key sk, ciphertext c, and outputs message m.

$\mathbf{Exp}^{\text{popriv1},b}_{\mathsf{Fn}}(\mathcal{A})$

$pp \xleftarrow{\$} \mathsf{Fn.Setup}(\lambda)$

$(pk, sk) \xleftarrow{\$} \mathsf{Fn.KeyGen}(pp)$

$b' \xleftarrow{\$} \mathcal{A}^{\textsc{Trans}}(pp)$

return b'

Oracle $\textsc{Trans}(t, x_0, x_1)$

$(req_0; st_0) \xleftarrow{\$} \mathsf{Fn.Req}(pk, t, x_0)$

$(req_1; st_1) \xleftarrow{\$} \mathsf{Fn.Req}(pk, t, x_1)$

$rep_0 \xleftarrow{\$} \mathsf{Fn.BlindEv}(sk, t, req_0)$

$rep_1 \xleftarrow{\$} \mathsf{Fn.BlindEv}(sk, t, req_1)$

$y_0 \leftarrow \mathsf{Fn.Finalize}(pk, t, rep_0; st_0)$

$y_1 \leftarrow \mathsf{Fn.Finalize}(pk, t, rep_1; st_1)$

$\tau \leftarrow (req_b, rep_b, y_0)$

$\tau' \leftarrow (req_{1-b}, rep_{1-b}, y_1)$

return (τ, τ')

Figure 9: Unlinkability experiment for POPRF Fn **under semi-honest server.**

$\mathbf{Exp}^{\text{SOM-Unp}}_{\mathsf{Fn}}(\mathcal{A})$

$\mathsf{q} \leftarrow \{\}$ // query counter table

$pp \leftarrow\!\!\$\ \mathsf{Fn.Setup}(\lambda)$

$(pk, sk) \leftarrow\!\!\$\ \mathsf{Fn.KeyGen}(pp)$

$(\gamma, [t_i, x_i, \sigma_i]_i{}^{\ell}) \leftarrow \mathcal{A}^{\text{BEv}}(pp, pk)$

require $(\forall_{i \neq j}^{\ell} x_i \neq x_j) \wedge (\mathsf{q}[\gamma] < \ell)$

return $[\sigma_i]_i^{\ell} = [\mathsf{Fn.Ev}(sk, t_i, x_i)]_i^{\ell}$

Oracle $\text{BEv}(t, Y)$

if $\mathsf{q}[t] = \perp$ **then** $\mathsf{q}[t] = 0$

$\mathsf{q}[t] \leftarrow \mathsf{q}[t] + 1$

return $\mathsf{Fn.BlindEv}(sk, t, Y)$

Figure 10: Strong one-more unpredictability experiment for POPRF Fn**.**

Correctness. The correctness of AE requires that for every secret key $sk \in \{0,1\}^{\text{kl}}$, every message $m \in \{0,1\}^*$, $\mathsf{AE.Dec}(sk, \mathsf{AE.Enc}(sk, m)) = m$.

Security. For random bit $b \in \{0,1\}$, we provide the security experiment $\mathbf{G}^{\text{AE},b}_{\text{AE}}$ for both confidentiality and authenticity in Figure 12.

We define the advantage function of an AE adversary $\mathcal{A}$ as follows,

$$\text{Adv}^{\text{AE}}_{\text{AE}}(\mathcal{A}) := \Pr\left[\mathbf{G}^{\text{AE},1}_{\text{AE}}(\mathcal{A}) = 1\right] - \Pr\left[\mathbf{G}^{\text{AE},0}_{\text{AE}}(\mathcal{A}) = 1\right].$$

Game $\mathbf{Exp}^{\text{bio-guess}}_{pp,\mathcal{W}}(\mathcal{A})$

$w \leftarrow\!\!\$\ \mathcal{W}(\perp)$

$\text{win} \leftarrow 0$

$\mathcal{A}^{\text{Test}}(pp)$

return win

Oracle $\text{Test}(i, w_i^*)$

Parse pp as $(pp_{\text{bio}}, \text{nos}, \text{sl})$

require $0 \leq i \leq \text{nos.sl} - 1$

$w_i \leftarrow [\,]$

for $j = 0$ to $\text{sl} - 1$ **do**

  $idx \leftarrow pp_{\text{bio}}[i][j]$

  $w_i[j] \leftarrow w[idx]$

if $w_i = w_i^*$ **then**

  $\text{win} \leftarrow 1$

return win

Figure 11: Biometric guessing experiment $\mathbf{Exp}^{\text{bio-guess}}_{pp,\mathcal{W}}(\mathcal{A})$.

Game $\mathbf{G}^{\text{AE},b}_{\text{AE}}(\mathcal{A})$

$sk \xleftarrow{\$} \{0,1\}^{\text{kl}}$

$b' \xleftarrow{\$} \mathcal{A}^{\text{Oenc,Odec}}$

return b'

Oracle $\text{Oenc}(m)$

$y_1 \xleftarrow{\$} \mathsf{AE.Enc}(sk, m)$

$y_0 \xleftarrow{\$} \{0,1\}^{|y_1|}$

return y_b

Oracle $\text{Odec}(c)$

require c is not output by $\text{Oenc}(\cdot)$

$x_1 \leftarrow \mathsf{AE.Dec}(sk, c)$

$x_0 \leftarrow \perp$

return x_b

Figure 12: Security game for authenticated encryption AE.

A.2 Pseudorandom Function Family (PRF)

We also utilize pseudorandom function (PRF) families in our protocol. A pseudorandom function family is a set of functions indexed by the secret key, $\{F_{sk}(\cdot)\}_{sk \in \{0,1\}^{\text{kl}}}$. For simplicity, we focus on PRF families with fixed message length ml, and output length the same as the key length kl. Namely, $F : \{0,1\}^{\text{kl}} \times \{0,1\}^{\text{ml}} \to \{0,1\}^{\text{kl}}$.

For random bit $b \in \{0,1\}$, we recap the PRF security game in Figure 13.

The advantage of a PRF adversary is defined as follows,

$$\text{Adv}^{\text{PRF}}_{F}(\mathcal{A}) := \Pr\left[\mathbf{G}^{\text{PRF},1}_{F}(\mathcal{A}) = 1\right] - \Pr\left[\mathbf{G}^{\text{PRF},0}_{F}(\mathcal{A}) = 1\right].$$

B Security Proofs

Notation. We prove the security of our protocol within the code-based game-playing framework. For every security game $\mathbf{G}$, we use $\Pr[\mathbf{G}]$ as shorthand for $\Pr[\mathbf{G}(\mathcal{A}) = 1]$, where $\mathcal{A}$ is the adversary playing game $\mathbf{G}$. For boolean variables, such as Bad, we use

Game $\mathrm{G}_F^{\mathrm{PRF},b}(\mathcal{A})$

$T \leftarrow \{\}$

$sk \xleftarrow{\$} \{0,1\}^{\mathrm{kl}}$

$b' \xleftarrow{\$} \mathcal{A}^{\textsc{New}}$

return b'

Oracle $\textsc{New}(m)$

if $T[m] = \bot$ **then**

  $T[m] \xleftarrow{\$} \{0,1\}^{\mathrm{kl}}$

$y_0 \leftarrow T[m]$

$y_1 \leftarrow F(sk, m)$

return y_b

Figure 13: Security game for PRF.

$\Pr[\mathrm{Bad}]$ to denote $\Pr[\mathrm{Bad} = \mathrm{true}]$. We ensure that the adversary $\mathcal{A}$ is specified in the context. For every game $\mathbf{G_i}$, we use $\mathbf{G_i'}$ to denote the game that is the same as $\mathbf{G_i}$ except that it sets the bad flag.

B.1 Proof for Generic BFRB protocol satisfying Client Message Indistinguishability and Integrity under Corrupted $\mathcal{S}$

Game $\mathbf{G_0}$. We let $\mathbf{G_0} = \mathbf{Exp}^{\text{sec-cor-}\mathcal{S}}_{\mathrm{BFRB},\mathcal{W}}(\mathcal{A}_{\text{sec-cor-}\mathcal{S}})$. The advantage of $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$ is

$$\mathrm{Adv}^{\text{sec-cor-}\mathcal{S}}_{\mathrm{BFRB},\mathcal{W}} = 2\Pr[\mathbf{G_0}] - 1.$$

Game $\mathbf{G_0'}$. As shown in Figure 14, we let game $\mathbf{G_0'}$ be the same as $\mathbf{G_0}$, except adding the checks on the output of Fn.Finalize to set the bad flag if the output is inconsistent with the correct one; for invalid evaluation $\bot$, the oracles LRVault and GetAuth return $\bot$ respectively, as in the original security game. Since this operation has no effect on the outputs of the game, it follows that

$$\Pr[\mathbf{G_0}] = \Pr[\mathbf{G_0'}].$$

Game $\mathbf{G_1}$. As shown in Figure 14, we let $\mathbf{G_1}$ be the same as $\mathbf{G_0}$, except that in the LRVault and GetAuth oracles, the outputs if are valid (not $\bot$), and is not consistent with that output from the honest execution of OEncBio on id using the POPRF key sk_{o} (Line 17 in Figure 2).

As a result, games $\mathbf{G_0'}$ and $\mathbf{G_1}$ are identical-until-bad. By applying the fundamental lemma of game playing [7], we have the following inequality,

$$\Pr[\mathbf{G_0'}] \le \Pr[\mathbf{G_1}] + \Pr[\mathbf{G_0'} \text{ sets bad}].$$

We now proceed to bound $\Pr[\mathbf{G_0'} \text{ sets bad}]$ using the soundness of the underlying POPRF. Let $\mathrm{Bad}_{\mathrm{LR}}$ denote the event that $\mathbf{G_0'}$ sets bad in LRVault oracle, and $\mathrm{Bad}_{\mathrm{auth}}$ for that in Oracle GetAuth. Then we have the following inequality by applying the union bound,

$$\Pr[\mathbf{G_0'} \text{ sets bad}] \le \Pr[\mathrm{Bad}_{\mathrm{LR}}] + \Pr[\mathrm{Bad}_{\mathrm{auth}}].$$

Since the conditions of setting $\mathrm{Bad}_{\mathrm{LR}}$ and $\mathrm{Bad}_{\mathrm{auth}}$ are the same, it suffices to analyze only one. Let us consider $\mathrm{Bad}_{\mathrm{LR}}$. The bad flag is set when the condition $w_{\mathrm{enc}}[i] \neq \bot \wedge w_{\mathrm{enc}}[i] \neq \mathsf{Fn.Ev}(sk_{\mathrm{o}}, \mathrm{id}\|i, w_i)$ (violating soundness) is met. Let us consider the soundness game in Figure 8. We construct an adversary $\mathcal{A}_{\text{ver-sound}}$ as follows. As this is the first reduction, we describe a detailed description of how adversary $\mathcal{A}_{\text{ver-sound}}$ acts as a challenger for $\mathbf{G_0'}$. For simplicity, we omit such descriptions in later reductions when they are clear from context. Adversary $\mathcal{A}_{\text{ver-sound}}$ starts by drawing a

$\mathbf{G_0'}$ $\mathbf{G_1}$ $\mathbf{G_2}$: Oracle $\mathsf{LRVault}(\mathrm{id}, q, \overline{w_{\mathrm{enc}}}, m_0, m_1)$

require $1 \le q \le \mathrm{q_B}[\mathrm{id}]$

if $MT[\mathrm{id}] \neq \bot$ **then return** $\bot$

$st_{\mathrm{c}} \leftarrow BT[\mathrm{id}, q]$

// $w_{\mathrm{enc}} \leftarrow \mathsf{UnblindBio}(pp, pk_{\mathrm{o}}, \overline{w_{\mathrm{enc}}}; st_{\mathrm{c}})$

// **if** $w_{\mathrm{enc}} = \bot$ **then return** $\bot$

$w_{\mathrm{enc}} \leftarrow []$

for $i = 0$ to $pp.\mathrm{nos} - 1$ **do**

  $w_{\mathrm{enc}}[i] \leftarrow \mathsf{Fn.Finalize}(pp, pk_{\mathrm{o}}, \overline{w_{\mathrm{enc}}}[i]; st_{\mathrm{c}}.st^i_{\mathrm{fn}})$

  if $w_{\mathrm{enc}}[i] = \bot$ **then**

    return $\bot$

  if $w_{\mathrm{enc}}[i] \neq \mathsf{Fn.Ev}(sk_{\mathrm{o}}, \mathrm{id}\|i, st_{\mathrm{c}}.w_i)$ **then**

    bad $\leftarrow$ true $\boxed{w_{\mathrm{enc}}[i] = \mathsf{Fn.Ev}(sk_{\mathrm{o}}, \mathrm{id}\|i, st_{\mathrm{c}}.w_i)}$

  $w_{\mathrm{enc}}[i] \leftarrow \mathsf{H}_1(\mathrm{id}\|i, st_{\mathrm{c}}.w_i, w_{\mathrm{enc}}[i])$

  $w_{\mathrm{enc}}[i] \xleftarrow{\$} \mathsf{RO}_1(\mathrm{id}\|i, st_{\mathrm{c}}.w_i, w_{\mathrm{enc}}[i])$

$V \leftarrow \mathsf{GenVault}(pp, \mathrm{id}, w_{\mathrm{enc}}, m_b)$

$MT[\mathrm{id}] \leftarrow m_b$

return V

$\mathbf{G_0'}$ $\mathbf{G_1}$ $\mathbf{G_2}$: Oracle $\mathsf{GetAuth}(\mathrm{id}, q, \overline{w_{\mathrm{enc}}})$

require $1 \le q \le \mathrm{q_B}[\mathrm{id}]$

$st_{\mathrm{c}} \leftarrow BT[\mathrm{id}, q]$

// $w_{\mathrm{enc}} \leftarrow \mathsf{UnblindBio}(pp, pk_{\mathrm{o}}, \overline{w_{\mathrm{enc}}}; st_{\mathrm{c}})$

// **if** $w_{\mathrm{enc}} = \bot$ **then return** $\bot$

$w_{\mathrm{enc}} \leftarrow []$

for $i = 0$ to $pp.\mathrm{nos} - 1$ **do**

  $w_{\mathrm{enc}}[i] \leftarrow \mathsf{Fn.Finalize}(pp, pk_{\mathrm{o}}, \overline{w_{\mathrm{enc}}}[i]; st_{\mathrm{c}}.st^i_{\mathrm{fn}})$

  if $w_{\mathrm{enc}}[i] = \bot$ **then**

    return $\bot$

  if $w_{\mathrm{enc}}[i] \neq \mathsf{Fn.Ev}(sk_{\mathrm{o}}, \mathrm{id}\|i, st_{\mathrm{c}}.w_i)$ **then**

    bad $\leftarrow$ true $\boxed{w_{\mathrm{enc}}[i] = \mathsf{Fn.Ev}(sk_{\mathrm{o}}, \mathrm{id}\|i, st_{\mathrm{c}}.w_i)}$

  $w_{\mathrm{enc}}[i] \leftarrow \mathsf{H}_1(\mathrm{id}\|i, st_{\mathrm{c}}.w_i, w_{\mathrm{enc}}[i])$

  $w_{\mathrm{enc}}[i] \xleftarrow{\$} \mathsf{RO}_1(\mathrm{id}\|i, st_{\mathrm{c}}.w_i, w_{\mathrm{enc}}[i])$

$ad \leftarrow \mathsf{GenAuthData}(pp, \mathrm{id}, w_{\mathrm{enc}})$

$\mathsf{AuthTable}[\mathrm{id}, ad] \leftarrow w_{\mathrm{enc}}$

return ad

$\mathbf{G_2}$: Oracle $\mathsf{RO}_1(t, x, \sigma)$

if $\mathsf{T}_{\mathsf{H}_1}[t, x, \sigma] = \bot$ **then**

  $\mathsf{T}_{\mathsf{H}_1}[t, x, \sigma] \xleftarrow{\$} \{0,1\}^{\lambda}$

return $\mathsf{T}_{\mathsf{H}_1}[t, x, \sigma]$

Figure 14: Games $\mathbf{G_0'}$, $\mathbf{G_1}$, and $\mathbf{G_2}$.

random bit $b \in \{0,1\}$ for $\mathbf{G_0'}$. It then samples a challenge biometric $w^* \xleftarrow{\$} \mathcal{W}(\bot)$. Instead of generating a new key pair for the POPRF, $\mathcal{A}_{\text{ver-sound}}$ uses the existing key pair (pk, sk) from its own soundness game. To generate the parameters at the start of the

game, $\mathcal{A}_{\text{ver-sound}}$ runs the ParamGen algorithm. Rather than generating new parameter pp_{Fn} for the POPRF, $\mathcal{A}_{\text{ver-sound}}$ reuses the parameter pp_{fn} from its own game (see Line 2 in Figure 8). Then let $pp \leftarrow (pp_{\text{bio}}, pp_{\text{fn}}, \text{fvl}, \text{nos}, \text{sl}, \text{maxctr})$, where the remaining parameters are fixed in the BFRB protocol. For each POPRF request made in the Blind oracle for a given id, $\mathcal{A}_{\text{ver-sound}}$ calls Req in its own game, which returns (req, st). The state st is stored in the blinding randomness table BT; for each Blind query, there are $pp.\text{nos}$ POPRF states. For every POPRF request resulting from the OEncBio call, the adversary $\mathcal{A}_{\text{ver-sound}}$ applies the POPRF secret key sk to evaluate the blinded POPRF request. The adversary $\mathcal{A}_{\text{ver-sound}}$ can perfectly simulate all oracles of $\mathbf{G}'_{\mathbf{0}}$.

We now bound the probability of Bad_{LR}. In the game $\mathbf{G}'_{\mathbf{0}}$, the adversary $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$ makes at most q_{LR} queries to the LRVault oracle for each id, and there are $pp.\text{nos}$ number of POPRF replies for each LRVault query. Suppose when $\mathcal{A}_{\text{ver-sound}}$ runs the game $\mathbf{G}'_{\mathbf{0}}$, the bad flag is set in LRVault at $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$'s j-th query and the k-th POPRF reply (i.e., $w_{\text{enc}}[k]$). Let $\mathcal{A}_{\text{ver-sound}}$ abort the game. Given the indices j and k, let $u = (j-1) \cdot pp.\text{nos} + k$, indicating that the u-th POPRF response evaluates to an incorrect POPRF output. Let $\mathcal{A}_{\text{ver-sound}}$ output $(u, \text{id} \| k, w_{\text{enc}}[k])$ as the incorrect POPRF reply in its POPRF soundness game. Due to the condition that sets the bad flag, this reply must violate the soundness guarantee of POPRF. Thus, $\mathcal{A}_{\text{ver}}$ wins the POPRF soundness game. We can bound $\Pr[\text{Bad}_{\text{LR}}]$ by union bound as follows.

$$\Pr[\text{Bad}_{\text{LR}}] \leq (q_{\text{LR}} \cdot pp.\text{nos}) \cdot \Pr\left[\mathbf{Exp}^{\text{VER-SOUND}}_{\mathsf{Fn}}(\mathcal{A}_{\text{ver-sound}}) = 1\right].$$

Given definition $\text{Adv}^{\text{VER-SOUND}}_{\mathsf{Fn}}(\mathcal{A}) := \Pr\left[\mathbf{Exp}^{\text{VER-SOUND}}_{\mathsf{Fn}}(\mathcal{A}) = 1\right]$, we obtain

$$\Pr[\text{Bad}_{\text{LR}}] \leq (q_{\text{LR}} \cdot pp.\text{nos}) \cdot \text{Adv}^{\text{VER-SOUND}}_{\mathsf{Fn}}(\mathcal{A}_{\text{ver-sound}}),$$

where $\mathcal{A}_{\text{ver-sound}}$ makes at most $q_{\text{LR}} \cdot pp.\text{nos}$ queries per id to its Req oracle.

Similarly, we can derive

$$\Pr[\text{Bad}_{\text{auth}}] \leq (q_{\text{auth}} \cdot pp.\text{nos}) \cdot \text{Adv}^{\text{VER-SOUND}}_{\mathsf{Fn}}(\mathcal{A}_{\text{ver-sound}}),$$

where $\mathcal{A}_{\text{ver-sound}}$ makes at most $q_{\text{auth}} \cdot pp.\text{nos}$ queries per id to its Req oracle.

Games $\mathbf{G_1} - \mathbf{G_5}$. For this series of game transitions, our goal is to replace every hashed w_{enc} with a random string of length λ. For different hashed subsamples w_{enc} (under a different id), these random strings should be independent.

This replacement could become distinguishable if the adversary inputs a w_{enc} into RO_1 that has been queried in either the LRVault or GetAuth oracles. Since the hash function is deterministic, the adversary can distinguish its output from a random string after the replacement. We bound this failure probability using two properties of the underlying POPRF: *unlinkability* and *strong one-more unpredictability*.

Consider how an adversary might attempt to produce such a w_{enc}: it could query OEncBio with different biometric subsamples and then test the outputs against the vault V or the authentication data ad, and thus win the LRVault challenge. However, the adversary can only achieve this through brute-force searching of the biometric subsamples. This is ensured by the unlinkability and strong one-more unpredictability of the POPRF. The former property ensures that the requests and replies from POPRF evaluations on secret subsamples reveal no partial information. The latter strong one-more unpredictability guarantees that the POPRF key is required to produce correct POPRF output, and each POPRF blinded evaluation results in only one valid POPRF output.

The rate-limiting strategy of the protocol effectively mitigates this risk by restricting the number of queries an adversary can make to OEncBio for a specific id within a certain time period. Furthermore, strong one-more unpredictability ensures that querying OEncBio for a particular id and subsamples does not help with producing w_{enc} for a different id without querying OEncBio. We also require the deterministic nature of Fn.Ev that guarantees the uniqueness of the POPRF output for each input (id, w_i), and the verifiability of POPRF. Together, these properties ensure these game transitions.

Game $\mathbf{G_2}$. We let game $\mathbf{G_2}$ be the same as game $\mathbf{G_1}$ except that $\mathbf{G_2}$ models H_1 as the random oracle, as shown in Figure 14. Since modeling H_1 as the random oracle is the only change,

$$\Pr[\mathbf{G_1}] = \Pr[\mathbf{G_2}].$$

Game $\mathbf{G_3}$. As shown in Figures 16 and 17, we let $\mathbf{G_3}$ be the same as $\mathbf{G_2}$ except that we use two tables, T_{R} and T_{H_1}, to track the values of the random oracle RO_1. For each new query to RO_1 made in either LRVault or GetAuth, a random value is assigned and stored in T_{R}. Meanwhile, T_{H_1} is used to track the values queried by the adversary to RO_1. If the adversary queries (t, x, σ) to RO_1, which has already been queried from LRVault or GetAuth, we resolve the output inconsistency by reprogramming RO_1 (i.e., updating the table T_{H_1}). Specifically, we ensure that T_{R} and T_{H_1} remain consistent with respect to this (t, x, σ). As a result of these repairs, the adversary's view of RO_1 is always consistent. It follows that,

$$\Pr[\mathbf{G_2}] = \Pr[\mathbf{G_3}].$$

Game $\mathbf{G_4}$. As shown in Figures 16 and 17, we let game $\mathbf{G_4}$ be the same as $\mathbf{G_3}$, except that we remove the repair operation after setting the bad flag. In addition, in the game $\mathbf{G_4}$, we introduce a query tracker q_{t} to the OEncBio oracle (see Figure 15). This tracker records the number of queries made to OEncBio with respect to some public tag t. At a high level, we aim to bound the probability that an adversary can produce a valid POPRF output σ beyond what could be derived from querying OEncBio. This probability is bounded by the strong one-more unpredictability of POPRF. If an adversary succeeds in producing such an output, we set the bad flag in RO_1. Similarly, under the same condition, we also set the bad flag in LRVault and GetAuth. The probability of the bad flag is being set can be bounded by constructing a strong one-more unpredictability adversary $\mathcal{A}_{\text{som-unp}}$ against the security game $\mathbf{Exp}^{\text{SOM-Unp}}_{\mathsf{Fn}}$ (see Figure 10). Let $\mathcal{A}_{\text{som-unp}}$ act as a challenger in the game $\mathbf{G_3}$ running by $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$. Similar to the reduction in $\mathbf{G_1}$, instead of generating new group parameters and key pair for the POPRF, $\mathcal{A}_{\text{som-unp}}$ uses the existing group parameters and key pair (pk, sk) from its own strong one-more unpredictability game. The remaining public parameters for biometrics are fixed as usual in the BFRB protocol.

For each POPRF request made in the Blind oracle for a given id, $\mathcal{A}_{\text{som-unp}}$ calls the Fn.Req of POPRF Fn, which returns (req, st). The state st is stored in the blinding randomness table BT; for each Blind

$\mathbf{G_4}$: Oracle OEncBio(id, $\overline{w}$)

17 // $(\overline{w_{\text{enc}}}; st_{\text{o}}) \leftarrow \text{EncBio}(pp, sk_{\text{o}}, \text{id}, \overline{w}; st_{\text{o}})$
$\overline{w_{\text{enc}}} \leftarrow [\,]$
if $st_{\text{o}}.\text{ctrs\_t}[\text{id}] = \perp$ **then**
$st_{\text{o}}.\text{ctrs\_t}[\text{id}] \leftarrow 0$
else
if $st_{\text{o}}.\text{ctrs\_t}[\text{id}] > pp.\text{maxctr}$ **then return** $\perp$
for $i = 0$ to $pp.\text{nos} - 1$ **do**
$t \leftarrow id \| i$; $\quad \text{q}_{\text{t}}[t] \leftarrow \text{q}_{\text{t}}[t] + 1$
$\overline{w_{\text{enc}}}[i] \xleftarrow{\$} \text{Fn.BlindEv}_{pp.pp_{\text{fn}}}(pp, sk_{\text{o}}, t, \overline{w}[i])$
18 **return** $\overline{w_{\text{enc}}}$

Figure 15: Game $\mathbf{G_4}$.

query, there are pp.nos POPRF states. For every POPRF blinded evaluation query made from OEncBio, the adversary $\mathcal{A}_{\text{som-unp}}$ calls its BEv oracle to evaluate the blinded POPRF request and outputs a POPRF reply, and forward it to $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$. The adversary $\mathcal{A}_{\text{ver-sound}}$ can perfectly simulate all oracles of the experiment.

When the bad flag is set, $\mathcal{A}_{\text{som-unp}}$ aborts the game $\mathbf{G_3}$ and outputs the set S (see Figure 16), which contains only valid POPRF outputs σ queried to RO_1. Since $\text{q}_{\text{t}}[t] < \text{cnt}[t]$, which means the number of valid POPRF output is POPRF queries evaluated under public tag t is, $\mathcal{A}_{\text{som-unp}}$ wins the strong one-more unpredictability game. Specifically, $\mathcal{A}_{\text{som-unp}}$ collects the set S, which contains a list of (t_i, x_i, σ_i), where σ_i is the valid POPRF output evaluated under POPRF sk and public tag t_i. $\mathcal{A}_{\text{som-unp}}$ filters S with t that sets the bad flag. Since $\text{Fn.Ev}(sk, t, x)$ is a deterministic function, $x_i \neq x_j$ for all x_i, x_j in S associated with t. The adversary $\mathcal{A}_{\text{som-unp}}$ outputs (t, S'). This wins the strong one-more unpredictability game.

Using the definition $\text{Adv}^{\text{SOM-UNP}}_{\text{Fn}}(\mathcal{A}) := \Pr\left[\mathbf{Exp}^{\text{SOM-UNP}}_{\text{Fn}}(\mathcal{A}) = 1\right]$, we claim the following inequality,

$$\Pr[\mathbf{G_3}] - \Pr[\mathbf{G_4}] \leq \Pr[\mathbf{G_3} \text{ sets bad}] \leq \text{Adv}^{\text{SOM-UNP}}_{\text{Fn}}(\mathcal{A}),$$

where $\mathcal{A}_{\text{som-unp}}$ makes $\text{q}_{\text{t}}[t]$ queries (which is at most $pp.\text{nos} \cdot q_{\text{B}}$) to its blinded evaluation oracle BEv for every queried public tag t in its own $\mathbf{Exp}^{\text{SOM-UNP}}_{\text{Fn}}$ game. The running time of $\mathcal{A}_{\text{som-unp}}$ is essentially the same as $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$.

Game $\mathbf{G_5}$ (Unlinkability). We define the game $\mathbf{G_5}$ to be the same as game $\mathbf{G_4}$, except that at the start of $\mathbf{G_5}$, in addition to generating the challenge biometric $w^* \xleftarrow{\$} \mathcal{W}(\perp)$, we generate an additional biometric $\widetilde{w} \xleftarrow{\$} \mathcal{W}(\perp)$. Later in $\mathbf{G_5}$, for each oracle call, $\widetilde{w}$ is used to generate the POPRF transcripts, while the challenge biometric w^* is used to generate POPRF outputs, as in $\mathbf{G_4}$.

Since the only difference between $\mathbf{G_4}$ and $\mathbf{G_5}$ lies in the POPRF transcripts, we bound $\Pr[\mathbf{G_4}] - \Pr[\mathbf{G_5}]$ through a reduction to the unlinkability property of the POPRF. Let $\widetilde{b}$ be the challenge bit for the unlinkability game POPRIV-1 played by $\mathcal{A}_{\text{popriv1}}$. Let $\mathcal{A}_{\text{popriv1}}$ be the challenger for $\mathbf{G_4}$, which draws a bit $b \in \{0, 1\}$, challenge biometric $w^* \xleftarrow{\$} \mathcal{W}(\perp)$ and an additional biometric $\widetilde{w} \xleftarrow{\$} \mathcal{W}(\perp)$. Since $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$ makes at most $pp.\text{nos} \cdot q_{\text{B}}$ POPRF queries for each id, we let $\mathcal{A}_{\text{popriv1}}$ query its TRANS oracle on subsamples of $\widetilde{w}$ and w^* with respect to the public tag $t = \text{id} \| i$, for $i \in [pp.\text{nos}]$, in order for q_{B} times. Each TRANS query yields $(rep_{\widetilde{b}}, rep_{\widetilde{b}}, y_0)$ and $(req_{1-\widetilde{b}}, rep_{1-\widetilde{b}}, y_1)$.

For each POPRF query made by $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$ in $\mathbf{G_4}$, we let $\mathcal{A}_{\text{popriv1}}$ forward the transcript $req_{\widetilde{b}}$, $rep_{\widetilde{b}}$, and the POPRF output y_1 in its own game to $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$. After $\mathbf{G_4}$ halts, $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$ outputs a bit b'. We let $\mathcal{A}_{\text{popriv1}}$ output 1 if $b' = b$, otherwise output 0. Since $\mathcal{A}_{\text{popriv1}}$ simulates $\mathbf{G_4}$ if $\widetilde{b} = 1$, and $\mathbf{G_5}$ if $\widetilde{b} = 0$, by union bound, we have the following inequality,

$$\Pr[\mathbf{G_4}] - \Pr[\mathbf{G_5}] \leq pp.\text{nos} \cdot q_{\text{B}} \cdot \text{Adv}^{\text{popriv1}}_{\text{Fn}}(\mathcal{A}_{\text{popriv1}}).$$

$\boxed{\mathbf{G_3}}\mathbf{G_4}$: $\mathbf{Exp}^{\text{sec-cor-}\mathcal{S}}_{\text{BFRB},\mathcal{W}}(\mathcal{A})$

$\vdots$
6 $\text{q}_{\text{B}}, \text{q}_{\text{E}}, \text{q}_{\text{LR}}, \text{q}_{\text{auth}} \leftarrow \{\}$ // counter tables
$\text{S} \leftarrow \{\}$ // Set for valid POPRF outputs
$\text{T}_{\text{H}_1} \leftarrow \{\}$ // table for random oracle RO_1
$pp \xleftarrow{\$} \text{ParamGen}(1^\lambda)$
$\vdots$

$\boxed{\mathbf{G_3}}\mathbf{G_4}$: Oracle LRVault(id, q, $\overline{w_{\text{enc}}}$, m_0, m_1)

19 **require** $1 \leq q \leq \text{q}_{\text{B}}[\text{id}]$
20 **if** $MT[\text{id}] \neq \perp$ **then return** $\perp$
if $\text{q}_{\text{LR}}[\text{id}] = \perp$ **then** $\text{q}_{\text{LR}}[\text{id}] = 0$
else $\text{q}_{\text{LR}}[\text{id}] = \text{q}_{\text{LR}}[\text{id}] + 1$
$\vdots$
21 $st_{\text{c}} \leftarrow BT[\text{id}, q]$
22 // $w_{\text{enc}} \leftarrow \text{UnblindBio}(pp, pk_{\text{o}}, \overline{w_{\text{enc}}}; st_{\text{c}})$
23 // **if** $w_{\text{enc}} = \perp$ **then return** $\perp$
$w_{\text{enc}} \leftarrow [\,]$
for $i = 0$ to $pp.\text{nos} - 1$ **do**
$w_{\text{enc}}[i] \leftarrow \text{Fn.Finalize}(pp, pk_{\text{o}}, \overline{w_{\text{enc}}}[i]; st_{\text{c}}.st^i_{\text{fn}})$
$w_i \leftarrow st_{\text{c}}.w_i$; $t_i \leftarrow \text{id} \| i$
if $\text{T}_{\text{R}}[t_i, w_i, w_{\text{enc}}[i]] = \perp$ **then**
$\text{T}_{\text{R}}[t_i, w_i, w_{\text{enc}}[i]] \xleftarrow{\$} \{0,1\}^\lambda$
if $\text{q}_{\text{t}}[t] < \text{cnt}[t]$ **then** ;
bad $\leftarrow$ true
$\boxed{\textbf{if } \text{T}_{\text{H}_1}[t_i, w_i, w_{\text{enc}}[i]] \neq \perp \textbf{ then}}$
$\boxed{\text{T}_{\text{R}}[t_i, w_i, w_{\text{enc}}[i]] \leftarrow \text{T}_{\text{H}_1}[t_i, w_i, w_{\text{enc}}[i]]}$
else
if $\text{T}_{\text{H}_1}[t_i, w_i, w_{\text{enc}}[i]] \neq \perp$ **then**
$\text{T}_{\text{R}}[t_i, w_i, w_{\text{enc}}[i]] \leftarrow \text{T}_{\text{H}_1}[t_i, w_i, w_{\text{enc}}[i]]$
$w_{\text{enc}}[i] \leftarrow \text{T}_{\text{R}}[t_i, w_i, w_{\text{enc}}[i]]$
24 $V \leftarrow \text{GenVault}(pp, \text{id}, w_{\text{enc}}, m_b)$
25 $MT[\text{id}] \leftarrow m_b$
26 **return** V

Figure 16: Games $\mathbf{G_3}$ and $\mathbf{G_4}$.

$\boxed{\mathbf{G_3}}\,\mathbf{G_4}$: Oracle $\mathrm{RO}_1(t, x, \sigma)$

44 **if** $\mathsf{q}_{\mathsf{H}_1}[t] \neq \bot$ **then** $\mathsf{q}_{\mathsf{H}_1}[t] \leftarrow 0$
45 **else** $\mathsf{q}_{\mathsf{H}_1}[t] \leftarrow \mathsf{q}_{\mathsf{H}_1}[t] + 1$
46 **if** $\mathsf{T}_{\mathsf{H}_1}[t, x, \sigma] = \bot$ **then** $\mathsf{T}_{\mathsf{H}_1}[t, x, \sigma] \xleftarrow{\$} \{0,1\}^\lambda$
47 **if** $\sigma = \mathsf{Fn.Ev}(sk, t, x)$ **then**
48 $\quad \mathsf{S} \leftarrow \mathsf{S} \cup \{(t, x, \sigma)\}$
49 $\quad$ **if** $\mathsf{cnt}[t] \neq \bot$ **then** $\mathsf{cnt}[t] \leftarrow 0$
50 $\quad$ **else** $\mathsf{cnt}[t] \leftarrow \mathsf{cnt}[t] + 1$
51 $\quad$ **if** $\mathsf{q}_t[t] < \mathsf{cnt}[t]$ **then**
52 $\quad\quad \mathsf{bad} \leftarrow \mathsf{true}$
53 $\quad\quad \boxed{\textbf{if } \mathsf{T}_{\mathsf{R}}[t, x, \sigma] \neq \bot \textbf{ then } \mathsf{T}_{\mathsf{H}_1}[t, x, \sigma] \leftarrow \mathsf{T}_{\mathsf{R}}[t, x, \sigma]}$
54 $\quad$ **else**
55 $\quad\quad$ **if** $\mathsf{T}_{\mathsf{R}}[t, x, \sigma] \neq \bot$ **then** $\mathsf{T}_{\mathsf{H}_1}[t, x, \sigma] \leftarrow \mathsf{T}_{\mathsf{R}}[t, x, \sigma]$
56 **return** $\mathsf{T}_{\mathsf{H}_1}[t, x, \sigma]$

Figure 17: Games $\mathbf{G_3}$ and $\mathbf{G_4}$ (continue).

Game $\mathbf{G'_5}$. As shown in Figure 18, we let game $\mathbf{G'_5}$ be the same as $\mathbf{G_5}$, except that it sets the bad flag. Since setting the flag does not change the game, we have

$$\Pr[\mathbf{G_5}] = \Pr\left[\mathbf{G'_5}\right].$$

Game $\mathbf{G'_6}$. Next, let $\mathbf{G'_6}$ be the same as $\mathbf{G'_5}$, except it removes the operation after setting the bad flag (see Figure 18). For simplicity, let Bad denote the event that the game $\mathbf{G'_5}$ sets the bad flag.

By the fundamental lemma of game playing, we have the following inequality,

$$\Pr\left[\mathbf{G'_5}\right] - \Pr\left[\mathbf{G'_6}\right] \leq \Pr[\mathsf{Bad}].$$

Game $\mathbf{G_6}$ (Min-entropy). Compared with game $\mathbf{G'_6}$, the game $\mathbf{G_6}$ removes setting the bad flag, and thus,

$$\Pr[\mathbf{G_6}] = \Pr\left[\mathbf{G'_6}\right].$$

Together we obtain the following inequality,

$$\Pr[\mathbf{G_5}] - \Pr[\mathbf{G_6}] \leq \Pr[\mathsf{Bad}].$$

We now bound $\Pr[\mathsf{Bad}]$. Let ρ denote the min-entropy of each subsample w_i of the challenge biometric w^*. We claim the following inequality,

$$\Pr[\mathsf{Bad}] \leq \frac{pp.\mathsf{nos} \cdot (q_{\mathsf{E}} - 1)}{2^\rho}.$$

If the bad flag is set to true, the term $pp.\mathsf{nos} \cdot (q_{\mathsf{E}} - 1)/2^\rho$ is due to the min-entropy of the subsamples and the union bound. Since there exists only one challenge biometric w^*, it suffices for $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$ to recover any POPRF output corresponding to any of the subsamples to win the security experiment $\mathbf{Exp}^{\text{sec-cor-}\mathcal{S}}_{\mathsf{BFRB},\mathcal{W}}$. In the game hop between $\mathbf{G_3}$ and $\mathbf{G_4}$, we established that for an adversary to find a POPRF output queried by the LRVault or GetAuth, it must query the OEncBio oracle. Given a client id id, each POPRF output related to a subsample is created under a different public tag $t = \mathsf{id}\|i$, for some subsample index i. For each OEncBio query made by the adversary, it can only make one guess for each subsample.

$\boxed{\mathbf{G'_5}}\,\mathbf{G'_6}$: Oracle $\mathsf{LRVault}(\mathsf{id}, q, \overline{w_{\mathsf{enc}}}, m_0, m_1)$

$\quad\vdots$
22 $\quad$ // $w_{\mathsf{enc}} \leftarrow \mathsf{UnblindBio}(pp, pk_{\mathrm{o}}, \overline{w_{\mathsf{enc}}}; st_{\mathrm{c}})$
23 $\quad$ // **if** $w_{\mathsf{enc}} = \bot$ **then return** $\bot$
$\quad w_{\mathsf{enc}} \leftarrow [\,]$
$\quad$ **for** $i = 0$ to $pp.\mathsf{nos} - 1$ **do**
$\quad\quad w_{\mathsf{enc}}[i] \leftarrow \mathsf{Fn.Finalize}(pp, pk_{\mathrm{o}}, \overline{w_{\mathsf{enc}}}[i]; st_{\mathrm{c}}.st^i_{\mathrm{fn}})$
$\quad\quad w_i \leftarrow st_{\mathrm{c}}.w_i;\ t_i \leftarrow \mathsf{id}\|i$
$\quad\quad$ **if** $\mathsf{T}_{\mathsf{R}}[t_i, w_i, w_{\mathsf{enc}}[i]] = \bot$ **then**
$\quad\quad\quad \mathsf{T}_{\mathsf{R}}[t_i, w_i, w_{\mathsf{enc}}[i]] \xleftarrow{\$} \{0,1\}^\lambda$
$\quad\quad$ **if** $\mathsf{q}_t[t] < \mathsf{cnt}[t]$ **then** ;
$\quad\quad$ **else** // $\mathsf{q}_t[t] \geq \mathsf{cnt}[t]$
$\quad\quad\quad$ **if** $\mathsf{T}_{\mathsf{H}_1}[t_i, w_i, w_{\mathsf{enc}}[i]] \neq \bot$ **then**
$\quad\quad\quad\quad \mathsf{bad} \leftarrow \mathsf{true}$
$\quad\quad\quad\quad \boxed{\mathsf{T}_{\mathsf{R}}[t_i, w_i, w_{\mathsf{enc}}[i]] \leftarrow \mathsf{T}_{\mathsf{H}_1}[t_i, w_i, w_{\mathsf{enc}}[i]]}$
$\quad\vdots$

$\boxed{\mathbf{G'_5}}\,\mathbf{G'_6}$: Oracle $\mathrm{RO}_1(t, x, \sigma)$

44 **if** $\mathsf{q}_{\mathsf{H}_1}[t] \neq \bot$ **then** $\mathsf{q}_{\mathsf{H}_1}[t] \leftarrow 0$
45 **else** $\mathsf{q}_{\mathsf{H}_1}[t] \leftarrow \mathsf{q}_{\mathsf{H}_1}[t] + 1$
46 **if** $\mathsf{T}_{\mathsf{H}_1}[t, x, \sigma] = \bot$ **then** $\mathsf{T}_{\mathsf{H}_1}[t, x, \sigma] \xleftarrow{\$} \{0,1\}^\lambda$
47 **if** $\sigma = \mathsf{Fn.Ev}(sk, t, x)$ **then**
48 $\quad \mathsf{S} \leftarrow \mathsf{S} \cup \{(t, x, \sigma)\}$
49 $\quad$ **if** $\mathsf{cnt}[t] = \bot$ **then** $\mathsf{cnt}[t] \leftarrow 1$
50 $\quad$ **else** $\mathsf{cnt}[t] \leftarrow \mathsf{cnt}[t] + 1$
51 $\quad$ **if** $\mathsf{q}_t[t] < \mathsf{cnt}[t]$ **then** ;
52 $\quad$ **else** // $\mathsf{q}_t[t] \geq \mathsf{cnt}[t]$
53 $\quad\quad$ **if** $\mathsf{T}_{\mathsf{R}}[t, x, \sigma] \neq \bot$ **then**
54 $\quad\quad\quad \mathsf{bad} \leftarrow \mathsf{true};$
55 $\quad\quad\quad \boxed{\mathsf{T}_{\mathsf{H}_1}[t, x, \sigma] \leftarrow \mathsf{T}_{\mathsf{R}}[t, x, \sigma]}$
56 **return** $\mathsf{T}_{\mathsf{H}_1}[t, x, \sigma]$

Figure 18: Games $\mathbf{G'_5}$ and $\mathbf{G'_6}$.

The probability of recovering a subsample with a single guess is $1/2^\rho$, where ρ is the min-entropy. The total number of guesses made by the adversary is at most $(q_{\mathsf{E}} - 1)$, as at least one query is required to create a challenge using LRVault. By applying the union bound over $pp.\mathsf{nos}$ subsamples, the probability that at least one subsample is successfully recovered, and thus the bad flag is set, is given by $pp.\mathsf{nos} \cdot (q_{\mathsf{E}} - 1)/2^\rho$.

Game $\mathbf{G_7}$. As shown in Figure 19, we let $\mathbf{G_7}$ be the same as $\mathbf{G_6}$, except that each PRF evaluation is replaced with a random string that is consistently sampled using oracle RSp. Let $\mathcal{A}_{\mathrm{prf}}$ be a PRF adversary and it draws a random bit b for simulation. Let $\widetilde{b}$ be the challenge bit in its own PRF game. When the adversary $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$ queries $F(\cdot, \cdot)$, let adversary $\mathcal{A}_{\mathrm{prf}}$ query its New oracle, and forward the reply to $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$. Hence, for $\widetilde{b} = 1$ and 0, $\mathcal{A}_{\mathrm{prf}}$ simulates $\mathbf{G_6}$

$\mathbf{G_6}$ $\boxed{\mathbf{G_7}}$ $\mathbf{G_8}$: Oracle LRVault(id, q, $\overline{w_{\text{enc}}}$, m_0, m_1)

```
    ⋮
22  // w_enc ← UnblindBio(pp, pk_o, w̄_enc; st_c)
23  // if w_enc = ⊥ then return ⊥
    w_enc ← []
    for i = 0 to pp.nos − 1 do
      w_enc[i] ← Fn.Finalize(pp, pk_o, w̄_enc[i]; st_c.st_fn^i)
      w_enc[i] ←$ {0,1}^λ
24  // V ← GenVault(pp, id, w_enc, m_b)
    V ← []
    for i = 0 to pp.nos − 1 do
      V[i].vid ← F(w_enc[i], ⟨0⟩)  [V[i].vid ←$ RSp(w_enc[i], ⟨0⟩)]
      vkey_i ← F(w_enc[i], ⟨1⟩)  [vkey_i ←$ RSp(w_enc[i], ⟨1⟩)]
      V[i].vval ← AE.Enc(vkey_i, m_b)
      cl ← |V[i].vval| ; V[i].vval ← {0,1}^cl
25  MT[id] ← m_b
26  return V
```

$\mathbf{G_6}$ $\boxed{\mathbf{G_7}}$ $\mathbf{G_8}$: Oracle GetAuth(id, q, $\overline{w_{\text{enc}}}$)

```
    ⋮
29  // w_enc ← UnblindBio(pp, pk_o, w̄_enc; st_c)
30  // if w_enc = ⊥ then return ⊥
    w_enc ← []
    for i = 0 to pp.nos − 1 do
      w_enc[i] ← Fn.Finalize(pp, pk_o, w̄_enc[i]; st_c.st_fn^i)
      w_enc[i] ←$ {0,1}^λ
31  // ad ← GenAuthData(pp, id, w_enc)
    ad ← []
    for i = 0 to pp.nos − 1 do
      ad[i] ← F(w_enc[i], ⟨0⟩)  [ad[i] ←$ RSp(w_enc[i], ⟨0⟩)]
32  AuthTable[id, ad] ← w_enc
33  return ad
```

$\boxed{\mathbf{G_7}}$: Oracle RSp(k, m)

```
if T_F[k, m] = ⊥ then
  T_F[k, m] ←$ {0,1}^λ
return T_F[k, m]
```

Figure 19: Games $\mathbf{G_6}$, $\mathbf{G_7}$ and $\mathbf{G_8}$.

and $\mathbf{G_7}$, respectively. When $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$ halts, it outputs a bit b'. Let $\mathcal{A}_{\text{prf}}$ return 1 if $b' = b$, otherwise return 0.

$$\Pr[\mathbf{G_6}] - \Pr[\mathbf{G_7}] \le pp.\text{nos} \cdot (q_{\text{LR}} + q_{\text{auth}}) \cdot \text{Adv}^{\text{PRF}}_{F}(\mathcal{A}_{\text{prf}}).$$

Game $\mathbf{G_8}$. As shown in Figure 19, we let $\mathbf{G_8}$ be the same as $\mathbf{G_7}$, except that each $V[i].vval$ is replaced with a random string. Let $\mathcal{A}_{\text{ae}}$ be an AE adversary that draws a random $b \in \{0,1\}$ for simulation. Let $\widetilde{b}$ be the challenge bit in its AE game. For each AE.Enc query made by $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$, we let $\mathcal{A}_{\text{ae}}$ query its own Oenc oracle and reply it to $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$. It simulates $\mathbf{G_7}$ and $\mathbf{G_8}$ for $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$ when $\widetilde{b} = 1$ and 0, respectively. Utilizing the AE security gives,

$$\Pr[\mathbf{G_7}] - \Pr[\mathbf{G_8}] \le pp.\text{nos} \cdot q_{\text{LR}} \cdot \text{Adv}^{\text{AE}}_{\text{AE}}(\mathcal{A}_{\text{ae}}).$$

$\mathbf{G'_8}$ $\boxed{\mathbf{G_9}}$: Oracle Test(id, V, ad)

```
34  if AuthTable[id, ad] = ⊥ then return ⊥
35  if MT[id] = ⊥ then return ⊥
36  else m ← MT[id]
37  if b = 1 then
38    w_enc ← AuthTable[id, ad]
      // V = (V[i], i) as in the syntax
39    // m' ← UnlockVault(w_enc, id, (V[i], i))
      c ← V[i].vval
      vkey_i ← F(w_enc[i], ⟨1⟩)
      m' ← AE.Dec(vkey_i, c)
      if m' ≠ ⊥ then
40      if m' ≠ m then
41        bad ← true; [return ⊥]
42        return 1
43  return ⊥
```

Figure 20: Games $\mathbf{G'_8}$ and $\mathbf{G_9}$.

Game $\mathbf{G'_8}$. We let game $\mathbf{G'_8}$ (see Figure 20) be the same as $\mathbf{G_8}$ (see Figure 19) except that it sets the bad flag.

$$\Pr[\mathbf{G_8}] = \Pr\left[\mathbf{G'_8}\right].$$

Game $\mathbf{G_9}$. We let game $\mathbf{G_9}$ be the same as $\mathbf{G'_8}$ except that it returns $\perp$ after the bad flag is set true (see Figure 20). Let Bad denote the event that game $\mathbf{G'_8}$ sets the bad flag. We bound the probability of Bad using AE security. We give an AE adversary $\mathcal{A}_{\text{ae}}$, who acts as a challenger that draws a random bit $b \in \{0,1\}$ in game $\mathbf{G'_8}$, run by $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$. For each encryption and decryption call made by the $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$, let $\mathcal{A}_{\text{ae}}$ call its Oenc and Odec oracles in its own game, and forward the replies to $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$. When the bad flag is set, abort $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$ and output (c, m'). Let $\mathcal{A}_{\text{ae}}$ query its Odec oracle on c. If the Odec oracle returns m', let $\mathcal{A}_{\text{ae}}$ output 1; otherwise 0. In this case, $\mathcal{A}_{\text{ae}}$ wins the AE game.

When the bad flag is set, it indicates that the $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$ has created a ciphertext that was not output by AE.Enc and decrypts successfully. Otherwise, decryption would yield m due to the decryption correctness of AE. The AE security of underlying authenticated encryption bounds this probability:

$$\Pr[\text{Bad}] \le pp.\text{nos} \cdot q_{\text{LR}} \cdot \text{Adv}^{\text{AE}}_{\text{AE}}(\mathcal{A}_{\text{ae}}).$$

Since games $\mathbf{G'_8}$ and $\mathbf{G_9}$ are identical-until-bad, by the fundamental lemma of game playing, we have the following inequality,

$$\Pr\left[\mathbf{G'_8}\right] - \Pr[\mathbf{G_9}] \le \Pr[\text{Bad}] \le pp.\text{nos} \cdot q_{\text{LR}} \cdot \text{Adv}^{\text{AE}}_{\text{AE}}(\mathcal{A}_{\text{ae}}).$$

In the game $\mathbf{G_9}$, the return argument is set to $\perp$ after the bad flag is set, and all game operations related to the challenge bit have been replaced. It follows that $\Pr[\mathbf{G_9}] = 1/2$.

We finally observe that per id, $\mathcal{A}_{\text{ver}}$ makes at most $pp.\text{nos}\cdot(q_{\text{LR}}+q_{\text{auth}})$ queries to its Req oracle in the soundness game, $\mathcal{A}_{\text{som-unp}}$ makes at most $pp.\text{nos}\cdot q_{\text{B}}$ queries to its own BEv oracle, $\mathcal{A}_{\text{popriv1}}$ makes at most $pp.\text{nos}\cdot q_{\text{B}}$ queries to its Trans oracle, $\mathcal{A}_{\text{prf}}$ makes at most $pp.\text{nos}\cdot(q_{\text{LR}}+q_{\text{auth}})$ queries to its New oracle, $\mathcal{A}_{\text{ae}}$ makes at most $2pp.\text{nos}\cdot q_{\text{LR}}$ queries to its Oenc oracle, at most $q_{\text{test}}+1$ queries to its Odec oracle. The running times of all adversaries are approximately the same as $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$.

B.2 Proof for Generic BFRB Satisfying Vault Security and Client Message Integrity under Corrupted $\mathcal{O}$

This proof is similar to the proof in Section B.1.

Game $\mathbf{G_0}$. We let $\mathbf{G_0} = \mathbf{Exp}^{\text{sec-cor-}\mathcal{O}}_{\text{BFRB},\mathcal{W}}(\mathcal{A}_{\text{sec-cor-}\mathcal{O}})$. The advantage of $\mathcal{A}_{\text{sec-cor-}\mathcal{O}}$ is

$$\text{Adv}^{\text{sec-cor-}\mathcal{O}}_{\text{BFRB},\mathcal{W}} = 2\Pr[\mathbf{G_0}] - 1.$$

Game $\mathbf{G_1}$. We let $\mathbf{G_1}$ be the same as $\mathbf{G_0}$, except that every POPRF output is replaced with honest PORPF output. Similar to the proof in Section B.1, we obtain

$$\Pr[\mathbf{G_1}]-\Pr[\mathbf{G_0}] \le pp.\text{nos}\cdot(q_{\text{auth}}+q_{\text{test}})\cdot\text{Adv}^{\text{VER-SOUND}}_{\text{Fn}}(\mathcal{A}_{\text{ver-sound}}),$$

where $\mathcal{A}_{\text{ver-sound}}$ makes at most $pp.\text{nos}\cdot(q_{\text{auth}}+q_{\text{test}})$ queries per id to its Req oracle.

Game $\mathbf{G_2}$. We let game $\mathbf{G_2}$ be the same as game $\mathbf{G_1}$ except that $\mathbf{G_2}$ models H_1 as the random oracle,

$$\Pr[\mathbf{G_1}] = \Pr[\mathbf{G_2}].$$

Game $\mathbf{G_3}$. We let $\mathbf{G_3}$ be the same as $\mathbf{G_2}$ except that we use two tables, T_{R} and $\mathsf{T}_{\mathsf{H}_1}$, to track the values of the random oracle RO_1. For each new query to RO_1 made in either GenAddVault or Test, a random value is assigned and stored in T_{R}. Meanwhile, $\mathsf{T}_{\mathsf{H}_1}$ is used to track the values queried by the adversary to RO_1. If the adversary queries (t, x, σ) to RO_1, which has already been queried from GenAddVault or Test, we resolve the output inconsistency by reprogramming RO_1 (i.e., updating the table $\mathsf{T}_{\mathsf{H}_1}$). Thus,

$$\Pr[\mathbf{G_2}] = \Pr[\mathbf{G_3}].$$

Game $\mathbf{G_4}$ (Unlinkability). We define the game $\mathbf{G_5}$ to be the same as game $\mathbf{G_4}$, except that at the start of $\mathbf{G_5}$, in addition to generating the challenge biometric $w^* \xleftarrow{\$} \mathcal{W}(\perp)$, we generate an additional biometric $\widetilde{w} \xleftarrow{\$} \mathcal{W}(\perp)$. Later in $\mathbf{G_5}$, for each oracle call, $\widetilde{w}$ is used to generate the POPRF transcripts, while the challenge biometric w^* is used to generate POPRF outputs, as in $\mathbf{G_3}$.

$$\Pr[\mathbf{G_3}] - \Pr[\mathbf{G_4}] \le pp.\text{nos}\cdot q_{\text{B}}\cdot\text{Adv}^{\text{popriv1}}_{\text{Fn}}(\mathcal{A}_{\text{popriv1}}).$$

Game $\mathbf{G_5}$. We let $\mathbf{G_5}$ be the same as $\mathbf{G_4}$, except that each PRF evaluation is replaced with a random string that is consistently sampled.

$$\Pr[\mathbf{G_4}] - \Pr[\mathbf{G_5}] \le pp.\text{nos}\cdot(q_{\text{vault}}+q_{\text{test}})\cdot\text{Adv}^{\text{PRF}}_{F}(\mathcal{A}_{\text{prf}}).$$

Game $\mathbf{G_6}$ (Min-entropy). Let $\mathbf{G_6}$ be the same as $\mathbf{G_5}$ except that it sets the bad flag if the adversary wins in the Auth. Let ρ denote the min-entropy of each subsample w_i of the challenge biometric w^*.

We claim the following inequality,

$$\Pr[\mathbf{G_5}] - \Pr[\mathbf{G_6}] \le \Pr[\text{Bad}] \le \max\left\{\frac{pp.\text{nos}\cdot q_{\text{auth}}}{2^{\rho}}, \frac{pp.\text{nos}\cdot q_{\text{auth}}}{2^{\lambda}}\right\}.$$

Since we assume the min-entropy of each subsample is smaller than the security parameter (otherwise, our protocol would not be necessary) $\rho \le \lambda$, the above equality becomes:

$$\Pr[\text{Bad}] \le \frac{pp.\text{nos}\cdot q_{\text{auth}}}{2^{\rho}}.$$

Suppose the bad flag is not set in $\mathbf{G_6}$, let us continue with the following game hops.

Game $\mathbf{G_7}$. We let $\mathbf{G_7}$ be the same as $\mathbf{G_6}$, except that each $V[i].vval$ is replaced with a random string.

$$\Pr[\mathbf{G_6}] - \Pr[\mathbf{G_7}] \le pp.\text{nos}\cdot q_{\text{vault}}\cdot\text{Adv}^{\text{AE}}_{\text{AE}}(\mathcal{A}_{\text{ae}}).$$

Game $\mathbf{G_8}$. We let game $\mathbf{G_8}$ be the same as $\mathbf{G_7}$ except that it returns $\perp$ after the bad flag is set true. When the bad flag is set, it indicates that the $\mathcal{A}_{\text{sec-cor-}\mathcal{O}}$ has created a ciphertext that was not output by AE.Enc and decrypts successfully. Otherwise, decryption would yield m due to the decryption correctness of AE. The AE security of underlying authenticated encryption bounds this probability:

$$\Pr[\mathbf{G_7}] - \Pr[\mathbf{G_8}] \le \Pr[\text{Bad}] \le pp.\text{nos}\cdot q_{\text{vault}}\cdot\text{Adv}^{\text{AE}}_{\text{AE}}(\mathcal{A}_{\text{ae}}).$$

In the game $\mathbf{G_8}$, the return argument is set to $\perp$ after the bad flag is set, and all game operations related to the challenge bit have been replaced. It follows that $\Pr[\mathbf{G_8}] = 1/2$.

We finally observe that per id, $\mathcal{A}_{\text{ver-sound}}$ makes at most $pp.\text{nos}\cdot(q_{\text{vault}}+q_{\text{test}})$ queries to its Req oracle in the soundness game, $\mathcal{A}_{\text{popriv1}}$ makes at most $pp.\text{nos}\cdot q_{\text{B}}$ queries to its Trans oracle, $\mathcal{A}_{\text{prf}}$ makes at most $pp.\text{nos}\cdot(q_{\text{vault}}+q_{\text{test}})$ queries to its New oracle, $\mathcal{A}_{\text{ae}}$ makes at most $2pp.\text{nos}\cdot q_{\text{vault}}$ queries to its Oenc oracle, at most $q_{\text{test}}+1$ queries to its Odec oracle. The running times of all adversaries are approximately the same as $\mathcal{A}_{\text{sec-cor-}\mathcal{O}}$.

B.3 Proof for Generic BFRB Satisfying Client Message Indistinguishability and Integrity under Corrupted $\mathcal{S}, \mathcal{O}$

Theorem 3. *Let $\mathcal{A}_{\text{sec-cor-}\mathcal{S}\text{-}\mathcal{O}}$ be a client message indistinguishability and integrity adversary against a generic BFRB protocol Π under corrupted storage server $\mathcal{S}$ and helper server $\mathcal{O}$ that makes at most $(q_{\text{B}}, q_{\text{LR}}, q_{\text{auth}}, q_{\text{test}})$ queries per* id *to its* Blind, LRVault, GetAuth, Test *oracles. Then we give a verifiability soundness adversary $\mathcal{A}_{\text{ver-sound}}$, an unlinkability (*POPRIV-1*) adversary $\mathcal{A}_{\text{popriv1}}$, a bio-guess adversary $\mathcal{A}_{\text{bio-guess}}$, a PRF adversary $\mathcal{A}_{\text{prf}}$, and an AE*

adversary $\mathcal{A}_{\text{ae}}$ *with the following advantage,*

$$\begin{aligned}
&\mathsf{Adv}^{\text{sec-cor-}\mathcal{S}\text{-}\mathcal{O}}_{\text{BFRB},\mathcal{W}}(\mathcal{A}_{\text{sec-cor-}\mathcal{S}\text{-}\mathcal{O}}) \\
&\leq 2pp.\text{nos} \cdot (q_{\text{LR}} + q_{\text{auth}}) \cdot \mathsf{Adv}^{\text{VER-SOUND}}_{\text{Fn}}(\mathcal{A}_{\text{ver-sound}}) \\
&+ 2pp.\text{nos} \cdot q_{\text{B}} \cdot \mathsf{Adv}^{\text{popriv1}}_{\text{Fn}}(\mathcal{A}_{\text{popriv1}}) \\
&+ 2\mathsf{Adv}^{\text{bio-guess}}_{pp,\mathcal{W}}(\mathcal{A}_{\text{bio-guess}}) \\
&+ 2pp.\text{nos} \cdot (q_{\text{LR}} + q_{\text{auth}}) \cdot \mathsf{Adv}^{\text{PRF}}_{F}(\mathcal{A}_{\text{prf}}) \\
&+ 4pp.\text{nos} \cdot q_{\text{LR}} \cdot \mathsf{Adv}^{\text{AE}}_{\text{AE}}(\mathcal{A}_{\text{ae}}),
\end{aligned}$$

where per id, $\mathcal{A}_{\text{ver-sound}}$ *makes at most* $pp.\text{nos} \cdot (q_{\text{LR}} + q_{\text{auth}})$ *queries to its* Req *oracle in the soundness game,* $\mathcal{A}_{\text{popriv1}}$ *makes at most* $pp.\text{nos} \cdot q_{\text{B}}$ *queries to its* Trans *oracle,* $\mathcal{A}_{\text{prf}}$ *makes at most* $pp.\text{nos} \cdot (q_{\text{LR}} + q_{\text{auth}})$ *queries to its* New *oracle,* $\mathcal{A}_{\text{ae}}$ *makes at most* $2pp.\text{nos} \cdot q_{\text{LR}}$ *queries to its* Oenc *oracle, at most* $q_{\text{test}} + 1$ *queries to its* Odec *oracle. The running times of all adversaries are approximately the same as* $\mathcal{A}_{\text{sec-cor-}\mathcal{S}\text{-}\mathcal{O}}$.

Game $\mathbf{G_0}$. We let $\mathbf{G_0} = \mathbf{Exp}^{\text{sec-cor-}\mathcal{S}\text{-}\mathcal{O}}_{\text{BFRB},\mathcal{W}}(\mathcal{A}_{\text{sec-cor-}\mathcal{S}\text{-}\mathcal{O}})$. The advantage of $\mathcal{A}_{\text{sec-cor-}\mathcal{S}\text{-}\mathcal{O}}$ is

$$\mathsf{Adv}^{\text{sec-cor-}\mathcal{S}\text{-}\mathcal{O}}_{\text{BFRB},\mathcal{W}} = 2\Pr[\mathbf{G_0}] - 1.$$

Game $\mathbf{G_1}$. As shown in Figure 14, we let $\mathbf{G_1}$ be the same as $\mathbf{G_0}$, except that every POPRF output is replaced with honest PORPF output. Similar to the proof in Section B.1, we obtain

$$\Pr[\mathbf{G_0}] - \Pr[\mathbf{G_1}] \leq pp.\text{nos} \cdot (q_{\text{LR}} + q_{\text{auth}}) \cdot \mathsf{Adv}^{\text{VER-SOUND}}_{\text{Fn}}(\mathcal{A}_{\text{ver-sound}}).$$

Game $\mathbf{G_2}$. We let game $\mathbf{G_2}$ be the same as game $\mathbf{G_1}$ except that $\mathbf{G_2}$ models H_1 as the random oracle,

$$\Pr[\mathbf{G_1}] = \Pr[\mathbf{G_2}].$$

Game $\mathbf{G_3}$. We let $\mathbf{G_3}$ be the same as $\mathbf{G_2}$ except that we use two tables, T_{R} and $\mathsf{T}_{\mathsf{H}_1}$, to track the values of the random oracle RO_1. For each new query to RO_1 made in either LRVault or GetAuth, a random value is assigned and stored in T_{R}. Meanwhile, $\mathsf{T}_{\mathsf{H}_1}$ is used to track the values queried by the adversary to RO_1. If the adversary queries (t, x, σ) to RO_1, which has already been queried from LRVault or GetAuth, we resolve the output inconsistency by reprogramming RO_1 (i.e., updating the table $\mathsf{T}_{\mathsf{H}_1}$).

$$\Pr[\mathbf{G_2}] = \Pr[\mathbf{G_3}].$$

Game $\mathbf{G_4}$ (Unlinkability). We define the game $\mathbf{G_5}$ to be the same as game $\mathbf{G_4}$, except that at the start of $\mathbf{G_5}$, in addition to generating the challenge biometric $w^* \xleftarrow{\$} \mathcal{W}(\perp)$, we generate a biometric $\widetilde{w} \xleftarrow{\$} \mathcal{W}(\perp)$. Later in $\mathbf{G_5}$, for each oracle call, $\widetilde{w}$ is used to generate the POPRF transcripts, while the challenge biometric w^* is used to generate POPRF outputs, as in $\mathbf{G_3}$.

$$\Pr[\mathbf{G_3}] - \Pr[\mathbf{G_4}] \leq pp.\text{nos} \cdot q_{\text{B}} \cdot \mathsf{Adv}^{\text{popriv1}}_{\text{Fn}}(\mathcal{A}_{\text{popriv1}}).$$

Game $\mathbf{G_5}$ (Min-entropy). We let $\mathbf{G_5}$ be the same as $\mathbf{G_4}$ except that it removes the operation after setting the bad flag (similar to Figure 18). Let ρ denote the min-entropy of each subsample w_i of the challenge biometric w^*. We claim the following inequality,

$$\Pr[\mathbf{G_4}] - \Pr[\mathbf{G_5}] \leq \mathsf{Adv}^{\text{bio-guess}}_{pp,\mathcal{W}}(\mathcal{A}_{\text{bio-guess}}).$$

Game $\mathbf{G_6}$. We let $\mathbf{G_6}$ be the same as $\mathbf{G_5}$, except that each PRF evaluation is replaced with a random string that is consistently sampled using oracle RSp.

$$\Pr[\mathbf{G_5}] - \Pr[\mathbf{G_6}] \leq pp.\text{nos} \cdot (q_{\text{LR}} + q_{\text{auth}}) \cdot \mathsf{Adv}^{\text{PRF}}_{F}(\mathcal{A}_{\text{prf}}).$$

Game $\mathbf{G_7}$. We let $\mathbf{G_7}$ be the same as $\mathbf{G_6}$, except that each $V[i].vval$ is replaced with a random string.

$$\Pr[\mathbf{G_6}] - \Pr[\mathbf{G_7}] \leq pp.\text{nos} \cdot q_{\text{LR}} \cdot \mathsf{Adv}^{\text{AE}}_{\text{AE}}(\mathcal{A}_{\text{ae}}).$$

Game $\mathbf{G_8}$. We let game $\mathbf{G_8}$ be the same as $\mathbf{G_7}$ except that it returns $\perp$ after the bad flag is set true (similar to Figure 20). Let Bad denote the event that the game $\mathbf{G_7}$ sets the bad flag. We bound the probability of Bad using AE security. When the bad flag is set, it indicates that the $\mathcal{A}_{\text{sec-cor-}\mathcal{O}}$ has created a ciphertext that was not output by AE.Enc and decrypts successfully. Otherwise, decryption would yield m due to the decryption correctness of AE. The AE security of underlying authenticated encryption bounds this probability:

$$\Pr[\mathbf{G_7}] - \Pr[\mathbf{G_8}] \leq \Pr[\text{Bad}] \leq pp.\text{nos} \cdot q_{\text{LR}} \cdot \mathsf{Adv}^{\text{AE}}_{\text{AE}}(\mathcal{A}_{\text{ae}}).$$

In the game $\mathbf{G_8}$, the return argument is set to $\perp$ after the bad flag is set, and all game operations related to the challenge bit have been replaced. It follows that $\Pr[\mathbf{G_8}] = 1/2$.

B.4 Security in the Case of an Authenticated Channel between $\mathcal{O}$ and $\mathcal{C}$

We start with the case of corrupted $\mathcal{S}$.

Theorem 4. *Assume an authenticated channel between the server* $\mathcal{O}$ *and clients. Let* $\mathcal{A}_{\text{sec-cor-}\mathcal{S}}$ *be an efficient adversary that makes at most* $(q_{\text{B}}, q_{\text{E}}, q_{\text{LR}}, q_{\text{auth}}, q_{\text{test}})$ *queries to its* Blind, OEncBio, LRVault, GetAuth, Test *oracles per* id. *Then we construct efficient adversaries* $\mathcal{A}_{\text{som-unp}}$, $\mathcal{A}_{\text{popriv1}}$, $\mathcal{A}_{\text{prf}}$, $\mathcal{A}_{\text{ae}}$ *such that*

$$\begin{aligned}
&\mathsf{Adv}^{\text{sec-cor-}\mathcal{S}}_{\text{BFRB},\mathcal{W}}(\mathcal{A}_{\text{sec-cor-}\mathcal{S}}) \\
&\leq 2pp.\text{nos} \cdot q_{\text{B}} \cdot \mathsf{Adv}^{\text{popriv1}}_{\text{Fn}}(\mathcal{A}_{\text{popriv1}}) \\
&+ \frac{pp.\text{nos} \cdot (q_{\text{E}} - 1)}{2^{\rho-1}} \\
&+ 2pp.\text{nos} \cdot (q_{\text{LR}} + q_{\text{auth}}) \cdot \mathsf{Adv}^{\text{PRF}}_{F}(\mathcal{A}_{\text{prf}}) \\
&+ 4pp.\text{nos} \cdot q_{\text{LR}} \cdot \mathsf{Adv}^{\text{AE}}_{\text{AE}}(\mathcal{A}_{\text{ae}}) \ .
\end{aligned}$$

The proof is the same as the one provided in Appendix B.1 except for skipping $\mathbf{G_1}$ for replacing with honest POPRF output. values. This is due to the honest evaluation now enforced by the oracles. Therefore, we can remove the advantage term for $\mathcal{A}_{\text{ver}}$ in the security theorem.

In the case where the $\mathcal{O}$ server is corrupted while the $\mathcal{S}$ server is honest, we require verifiability of the POPRF only during the Register phase, not during the Retrieve phase. However, we require additional *key robustness* of authenticated encryption used for encrypting the vaults. This modification affects our theorem and has implications for the Test oracle. Since we do not require the verifiability of the POPRF during the retrieval phase, the POPRF outputs w_{enc} may differ from those generated during registration. It is still possible for a different hashed w_{enc}, used as the PRF key for authentication data, to generate the same authentication data as in the Register phase. In this scenario, the authentication succeeds

and likely produces a different vault key (under a different PRF key for the input bitstring $\langle 1 \rangle$). If the adversary wins the integrity game, it would violate the key robustness. We rely on the same justification as that provided in OPAQUE [33]. This gives us the following Theorem 5.

Theorem 5. *Assume an authenticated channel between the O server and clients. Let $\mathcal{A}_{\text{sec-cor-}O}$ be an efficient adversary that makes at most $(q_B, q_{\text{vault}}, q_{\text{auth}}, q_{\text{test}})$ queries per* id *to its* Blind, GenAddVault, Auth, Test *oracles. We then construct efficient soundness adversary $\mathcal{A}_{\text{ver-sound}}$, unlinkability (*POPRIV-1*) adversary $\mathcal{A}_{\text{popriv1}}$, PRF adversary $\mathcal{A}_{\text{prf}}$, and AE adversary (with key robustness) $\mathcal{A}_{\text{ae}}$ such that*

$$\begin{aligned}
&\mathsf{Adv}^{\text{sec-cor-}O}_{\text{BFRB},\mathcal{W}}(\mathcal{A}_{\text{sec-cor-}O}) \\
&\leq 2pp.\text{nos} \cdot q_{\text{vault}} \cdot \mathsf{Adv}^{\text{VER-SOUND}}_{\text{Fn}}(\mathcal{A}_{\text{ver-sound}}) \\
&+ 2pp.\text{nos} \cdot q_B \cdot \mathsf{Adv}^{\text{popriv1}}_{\text{Fn}}(\mathcal{A}_{\text{popriv1}}) + \frac{pp.\text{nos} \cdot q_{\text{auth}}}{2^{\rho-1}} \\
&+ 2pp.\text{nos} \cdot (q_{\text{vault}} + q_{\text{test}}) \cdot \mathsf{Adv}^{\text{PRF}}_{F}(\mathcal{A}_{\text{prf}}) \\
&+ 4pp.\text{nos} \cdot q_{\text{vault}} \cdot \mathsf{Adv}^{\text{AE}}_{\text{AE}}(\mathcal{A}_{\text{ae}}) \ .
\end{aligned}$$

In the case of both servers colluding, we can directly apply most of the proof for Theorem 3, along with the same security argument provided for Theorem 5. We require the key robustness of the authenticated encryption. This gives us the following Theorem 6.

Theorem 6. *Assume an authentication channel between the client C and O server. Let $\mathcal{A}_{\text{sec-cor-}\mathcal{S}\text{-}O}$ be a client message indistinguishability and integrity adversary against a generic BFRB protocol Π under corrupted storage server $\mathcal{S}$ and helper server O that makes at most $(q_B, q_{\text{LR}}, q_{\text{auth}}, q_{\text{test}})$ queries per* id *to its* Blind, LRVault, GetAuth, Test *oracles. Then we give a verifiability soundness adversary $\mathcal{A}_{\text{ver-sound}}$, an unlinkability (*POPRIV-1*) adversary $\mathcal{A}_{\text{popriv1}}$, a bio-guess adversary $\mathcal{A}_{\text{bio-guess}}$, a PRF adversary $\mathcal{A}_{\text{prf}}$, and an AE adversary $\mathcal{A}_{\text{ae}}$ (with key robustness) with the following advantage,*

$$\begin{aligned}
&\mathsf{Adv}^{\text{sec-cor-}\mathcal{S}\text{-}O}_{\text{BFRB},\mathcal{W}}(\mathcal{A}_{\text{sec-cor-}\mathcal{S}\text{-}O}) \\
&\leq 2pp.\text{nos} \cdot q_{\text{LR}} \cdot \mathsf{Adv}^{\text{VER-SOUND}}_{\text{Fn}}(\mathcal{A}_{\text{ver-sound}}) \\
&+ 2pp.\text{nos} \cdot q_B \cdot \mathsf{Adv}^{\text{popriv1}}_{\text{Fn}}(\mathcal{A}_{\text{popriv1}}) \\
&+ 2\mathsf{Adv}^{\text{bio-guess}}_{pp,\mathcal{W}}(\mathcal{A}_{\text{bio-guess}}) \\
&+ 2pp.\text{nos} \cdot (q_{\text{LR}} + q_{\text{auth}}) \cdot \mathsf{Adv}^{\text{PRF}}_{F}(\mathcal{A}_{\text{prf}}) \\
&+ 4pp.\text{nos} \cdot q_{\text{LR}} \cdot \mathsf{Adv}^{\text{AE}}_{\text{AE}}(\mathcal{A}_{\text{ae}}),
\end{aligned}$$

where per id, $\mathcal{A}_{\text{ver-sound}}$ makes at most $pp.\text{nos} \cdot q_{\text{LR}}$ queries to its Req *oracle in the soundness game, $\mathcal{A}_{\text{popriv1}}$ makes at most $pp.\text{nos} \cdot q_B$ queries to its* Trans *oracle, $\mathcal{A}_{\text{prf}}$ makes at most $pp.\text{nos} \cdot (q_{\text{LR}} + q_{\text{auth}})$ queries to its* New *oracle, $\mathcal{A}_{\text{ae}}$ makes at most $2pp.\text{nos} \cdot q_{\text{LR}}$ queries to its* Oenc *oracle, at most $q_{\text{test}}+1$ queries to its* Odec *oracle. The running times of all adversaries are approximately the same as $\mathcal{A}_{\text{sec-cor-}\mathcal{S}\text{-}O}$.*

B.5 Proof for POPRF 3H-POPRF Satisfying Strong One-More Unpredictability

We prove that POPRF 3H-POPRF construction in [38] satisfies the strong one-more unpredictability given Figure 10. Our proof is based on the one-more gap strong Diffie-Hellman inversion assumption given in the same work [38].

Game $\mathbf{G}^{(m,n)\text{-OM-Gap-SDHI}}_{\lambda}(\mathcal{A})$

1 $\mathsf{q} \leftarrow \{\}$ // query counter table
2 $(p, g, \mathbb{G}) \leftarrow\!\$\ \mathsf{GGen}(\lambda)$
3 $sk \leftarrow\!\$\ \mathbb{Z}_p$
4 $[y_i]^m_i \leftarrow\!\$\ [\mathbb{Z}_p]^m_i$
5 $(st_{\mathcal{A}}, [c_i]^n_i) \leftarrow\!\$\ \mathcal{A}_1(p, \mathbb{G})$
6 **require** $\forall^n_{i \neq j} c_i \neq c_j$
7 $(\gamma, [Z_i, \alpha_i]^{\ell}_i) \leftarrow\!\$\ \mathcal{A}_2^{\text{SDH,SDDH}}(g, g^{sk}, [g^{y_i}]^m_i; st_{\mathcal{A}})$
8 **require** $\mathsf{q}[\gamma] < \ell \wedge \forall^{\ell}_{i \neq j} \alpha_i \neq \alpha_j$
9 **return** $[Z_i]^{\ell}_i = \left[g^{y_{\alpha_i}/(sk+c_\gamma)}\right]^{\ell}_i$

Oracle $\mathsf{SDH}(B, i)$

10 **require** $i \in [n]$
11 **if** $\mathsf{q}[i] = \perp$ **then** $\mathsf{q}[i] = 0$
12 $\mathsf{q}[i] \leftarrow \mathsf{q}[i] + 1$
13 $Z \leftarrow B^{1/(sk+c_i)}$
14 **return** Z

Oracle $\mathsf{SDDH}(Y, Z, i)$

15 **return** $Z = Y^{1/(sk+c_i)}$

Figure 21: The one-more gap strong Diffie-Hellman inversion security game.

First, let us recall the construction of 3H-POPRF Fn. Let $\mathbb{G}$ be a group of prime order p, having a generator g. Similar to [38], we define three hash functions: $\mathsf{H}_1 : \{0,1\}^* \rightarrow \mathbb{G}$, $\mathsf{H}_2 : \{0,1\}^* \rightarrow \{0,1\}^{\lambda}$, and $\mathsf{H}_3 : \{0,1\}^* \rightarrow \{0,1\}^{2\lambda}$. Let NiZK (non-interactive zero-knowledge proof) uses the relation $\mathcal{R} = \{(g, U, V, W), (\alpha) : U = g^{\alpha}\}$. 3H-POPRF contains the following algorithms and the algorithms specified in Figure 22.

- $pp \xleftarrow{\$} \mathsf{Fn.Setup}(\lambda)$: $(p, g, \mathbb{G}) \xleftarrow{\$} \mathsf{GGen}(\lambda)$; $pp \leftarrow (p, g, \mathbb{G})$.
- $(pk, sk) \xleftarrow{\$} \mathsf{Fn.KeyGen}(pp)$: $(p, g, \mathbb{G}) \leftarrow pp$; $sk \xleftarrow{\$} \mathbb{Z}_p$; $pk \leftarrow g^{sk}$.
- $y \leftarrow \mathsf{Fn.Ev}(sk, t, x)$: $\mathsf{Fn.Ev}(sk, t, x) = \mathsf{H}_2(t, x, \mathsf{H}_1(x)^{1/(sk+\mathsf{H}_3(t))})$.

Before presenting the proof, we first recap the (m, n)-OM-Gap-SDHI security game [38], as shown in Figure 21. We adapt their syntax to explicitly include the state, denoted as $st_{\mathcal{A}}$, and change the notation of the query counter. Let $\mathcal{A}$ be an adversary against the (m, n)-OM-Gap-SDHI security game. It is a two-staged adversary: $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, with $\mathcal{A}_1$ choosing n messages $[c_i]^n_i$. There are m random messages $[y_i]^m_i$ chosen randomly from $\mathbb{Z}_p$.

The advantage of an adversary $\mathcal{A}$ against $\mathbf{G}^{(m,n)\text{-OM-Gap-SDHI}}_{\lambda}$ is defined as,

$$\mathsf{Adv}^{(m,n)\text{-OM-Gap-SDHI}}_{\lambda}(\mathcal{A}) := \Pr\left[\mathbf{G}^{(m,n)\text{-OM-Gap-SDHI}}_{\lambda}(\mathcal{A}) = 1\right].$$

Theorem 7 (Strong One-More Unpredictability of 3H-POPRF). *Let $\mathcal{A}$ be an adversary against $\mathbf{Exp}^{\text{SOM-UNP}}_{\text{Fn}}$ that makes at most m queries to H_1 and n queries to H_3. We construct an adversary $\mathcal{B}^{\mathcal{A}}$*

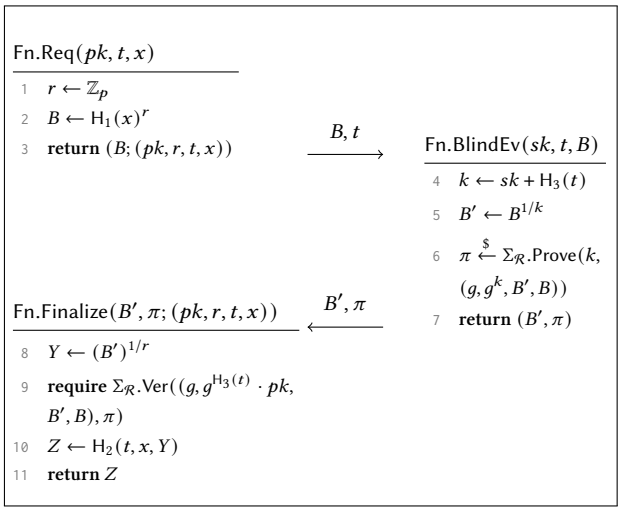

Figure 22: Algorithms Fn.Req**,** Fn.BlindEv **and** Fn.Finalize **of** 3H-POPRF**.**

against $\mathbf{G}_{\lambda}^{(m,n)\text{-OM-Gap-SDHI}}$ *with the following advantage,*

$$\mathsf{Adv}_{\mathsf{Fn}}^{\text{SOM-UNP}}(\mathcal{A}) \leq \mathsf{Adv}_{\lambda}^{(m,n)\text{-OM-Gap-SDHI}}(\mathcal{B}) + \frac{1}{2^{\lambda}},$$

where the $\mathcal{B}$ *makes at most* n *queries to its* SDH *oracle and* m *queries to its* SDDH *oracle, and the running time of* $\mathcal{B}$ *is almost the same as* $\mathcal{A}$*.*

Proof. We model the hash functions $\mathsf{H}_1, \mathsf{H}_2, \mathsf{H}_3$ as random oracles $\mathsf{RO}_1, \mathsf{RO}_2, \mathsf{RO}_3$, respectively. We construct an adversary $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ for the game $\mathbf{G}_{\lambda}^{(m,n)\text{-OM-Gap-SDHI}}$ as follows. The adversary $\mathcal{B}$ acts as a challenger in the game $\mathbf{Exp}_{\mathsf{Fn}}^{\text{SOM-UNP}}$ played by $\mathcal{A}$. At the start of the game $\mathbf{G}_{\lambda}^{(m,n)\text{-OM-Gap-SDHI}}$, $[y_i]_i^m$ is sampled independently and uniformly at random from $\mathbb{Z}_p$. Let adversary $\mathcal{B}_1$ output n messages $[c_i]_i^n$ independently and uniformly at random from $\mathbb{G}$. Let $\mathbf{Exp}_{\mathsf{Fn}}^{\text{SOM-UNP}}$ played by $\mathcal{A}$ uses the same group parameter as $\mathcal{B}$ in its own game $\mathbf{G}_{\lambda}^{(m,n)\text{-OM-Gap-SDHI}}$.

For each x queried by $\mathcal{A}$ to RO_1: if x has not been queried before, we assume it is the i-th query and set $\mathsf{T}_{\mathsf{H}_1}[x] \leftarrow g^{y_i}$. Otherwise, we let RO_1 return $\mathsf{T}_{\mathsf{H}_1}[x]$. We assume that $\mathcal{A}$ makes at most m queries for private input x to RO_1. Similarly, we use $\mathsf{T}_{\mathsf{H}_3}$ to track the values in RO_3. For each t queried by $\mathcal{A}$ to $\mathsf{T}_{\mathsf{H}_3}$: if t has not been queried before, we assume it is the j-th query and set $\mathsf{T}_{\mathsf{H}_3}[t] \leftarrow (c_j, j)$, returning c_j. Otherwise, we let RO_3 return the first element of $\mathsf{T}_{\mathsf{H}_3}[t]$.

For every BEv query (t, Y) made by $\mathcal{A}$: the public tag t is input to RO_3. RO_3 either returns a new c_j or an existing c_i, where c_i corresponds to t for some message stored in $\mathsf{T}_{\mathsf{H}_3}$. We let $\mathcal{B}_2$ query its own SDH oracle with (Y, i) and forward the oracle's reply to $\mathcal{A}$. As the SDH oracle performs the same operation as BEv, $\mathcal{B}$ can perfectly simulate the BEv oracle.

To create a valid POPRF output, one must either guess randomly from the output domain $\{0,1\}^{\lambda}$, or query $\mathsf{RO}_2(\cdot)$. In the first case, the success probability is $1/2^{\lambda}$. For the second case, we use reduction. Assuming that τ is stored in the table of RO_2, adversary $\mathcal{B}$ can simulate the equality check $\sigma = \mathsf{Fn.Ev}(sk, t, x)$ by utilizing its SDDH oracle. On input (sk, t, x), $\mathcal{B}$ first obtains the message index i of t, corresponding to c_i, using $\mathsf{T}_{\mathsf{H}_1}$. It then queries SDDH with $(\mathsf{T}_{\mathsf{H}_1}(x), \tau, i)$. If true is returned then σ is a correct POPRF output. When the game $\mathbf{Exp}_{\mathsf{Fn}}^{\text{SOM-UNP}}$ halts, adversary $\mathcal{A}$ outputs $(\gamma, [t_i, x_i, \sigma_i]_i^{\ell})$, where all σ_i are valid outputs with respect to Fn.Ev (simulated by $\mathcal{B}$).

If $\mathcal{A}$ is able to win the strong one-more unpredictability game, it produces an output satisfying the condition $(\forall_{i \neq j}^{\ell} x_i \neq x_j) \wedge (\mathsf{q}[\gamma] < \ell)$, and $[\sigma_i]_i^{\ell} = [\mathsf{Fn.Ev}(sk, t_i, x_i)]_i^{\ell}$. We let $\mathcal{B}$ output $(\gamma, [\tau_i, \alpha_i]_i^{\ell})$ in its own game, where $\tau_i = \mathsf{H}_2(t_i, x_i, \sigma_i)$. Given the advantage function, we conclude the proof. □

Remark 1. Note it is straightforward to observe that, in the proof, the inner part of 3H-POPRF already satisfies the one-more unpredictability. Hence, in the instantiation of the BFRB protocol, we can avoid one extra hashing step; directly using the 3H-POPRF is sufficient.